Introduction to Artificial Intelligence

Chapter 6 - Game Trees

Namyup Kim

Soongsil University



Source: UC Berkeley CS188, slides by Dan Klein and Pieter Abbeel (ai.berkeley.edu)

Recap of Chapter 5

- A constraint satisfaction problem is
  - Variables, Domains and Constraints
  - Its goal test (*constraint*) is something an algorithm can read during searching.
- Backtracking search is depth-first with one variable at a time and the constraints checked as you go
- Filtering and ordering are what make it fast ; min-conflicts often beats all of it.
- But every problem until ***Ch.5*** had a single agent in it.



*Today there are **Two Agents**, and the second one is trying to make you lose.*



Contents

- Adversarial games: what a game is, written down formally, and what zero-sum buys you
- Adversarial search: the value of a state when somebody else controls half the moves, and the minimax algorithm
- Game tree pruning: alpha-beta, which gives the same answer for far less work
- Resource limits and evaluation functions: what to do when you cannot reach the end of the game
- Uncertain outcomes: "*expectimax*", where the opponent is replaced by an average
- Utilities and rationality — what the numbers we average actually are, and whether we are allowed to average them

Learning Objectives

- **Write a game down formally**: states, players, actions, terminal test, terminal utilities and say what the zero-sum assumption buys you
- Compute **minimax values** by hand, and run **alpha-beta pruning** on a tree, including saying what it does and does not tell you afterwards
- Explain why a depth limit forces you to invent an **evaluation function**, and design one for a given game
- Choose between minimax and **expectimax** by looking at the environment, and say what each assumption costs when it turns out to be wrong
- State what a utility is, why averaging needs more than an ordering, and what the axioms of rationality buy you

Introduction to Artificial Intelligence

CS188 6 - Part I: Adversarial Search

Namyup Kim

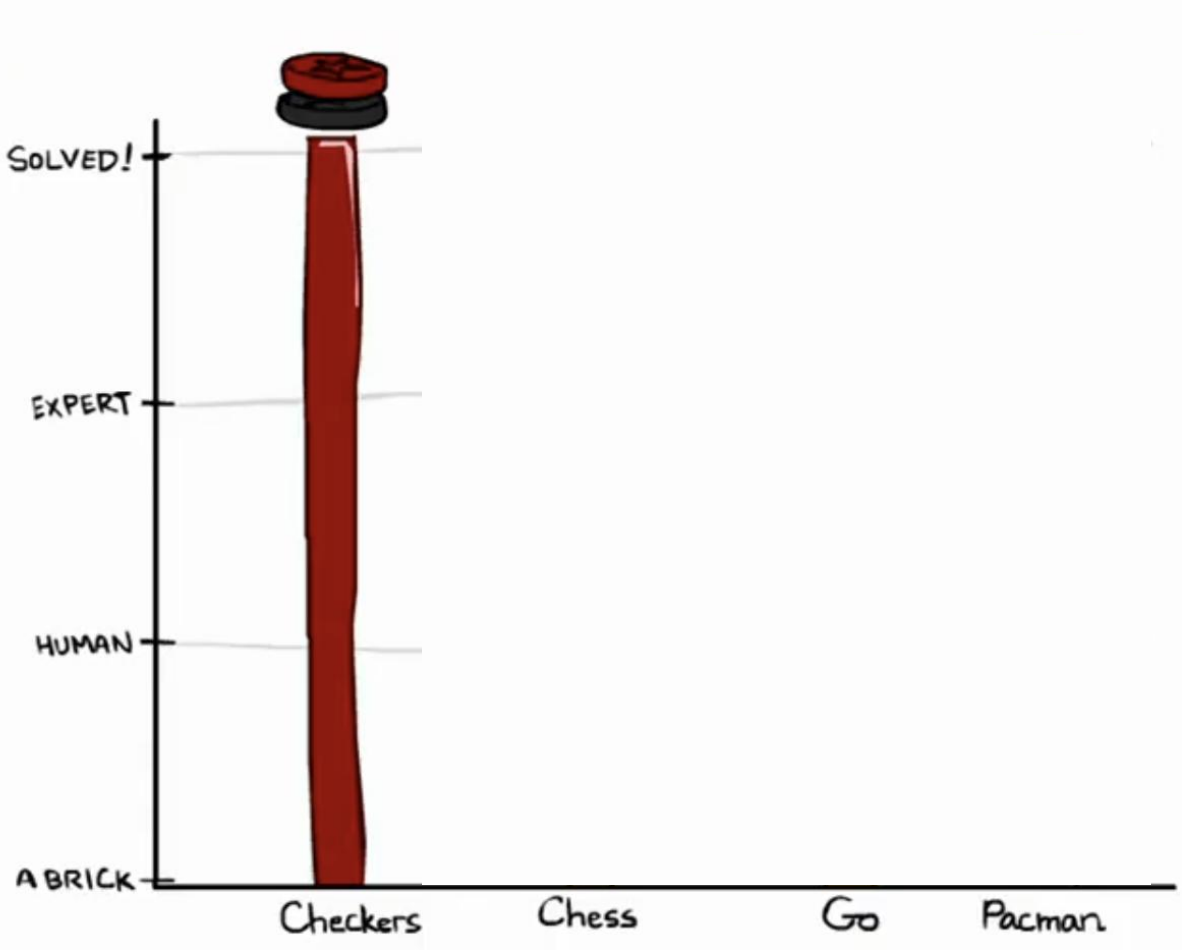
Soongsil University



Source: UC Berkeley CS188, slides by Dan Klein and Pieter Abbeel (ai.berkeley.edu)

Game Playing State-of-the-Art

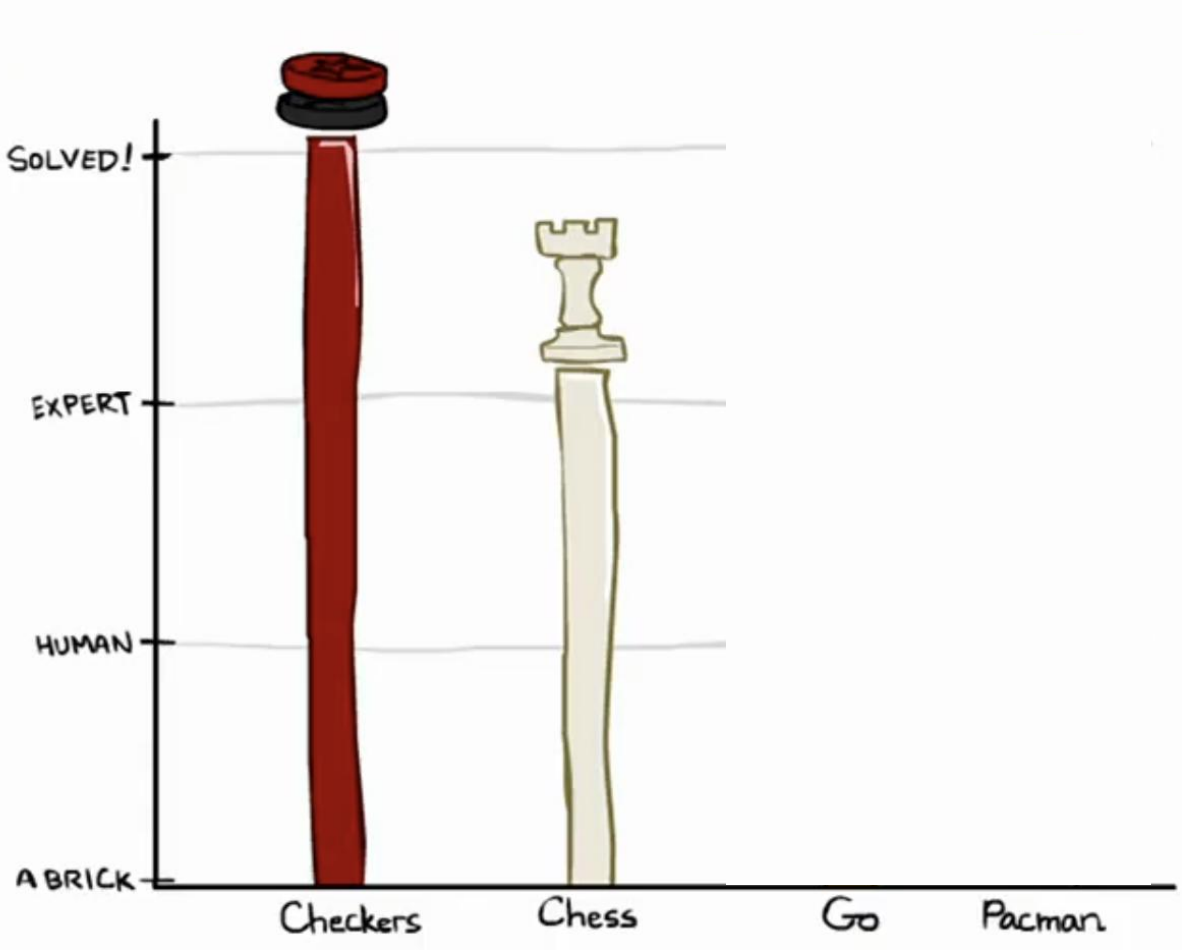
- CHECKERS:
  - 1950: the first computer player.
  - 1994: Chinook ends Marion Tinsley's forty-year dynasty as human champion.
  - 2007: checkers is SOLVED
  - SOLVED means **we know the whole game tree**. Perfect play from the start is a draw.



Game Playing State-of-the-Art

- CHECKERS:
  - 1950: the first computer player.
  - 1994: Chinook ends Marion Tinsley's forty-year dynasty as human champion.
  - 2007: checkers is SOLVED
  - **SOLVED means we know the whole game tree**. Perfect play from the start is a draw.

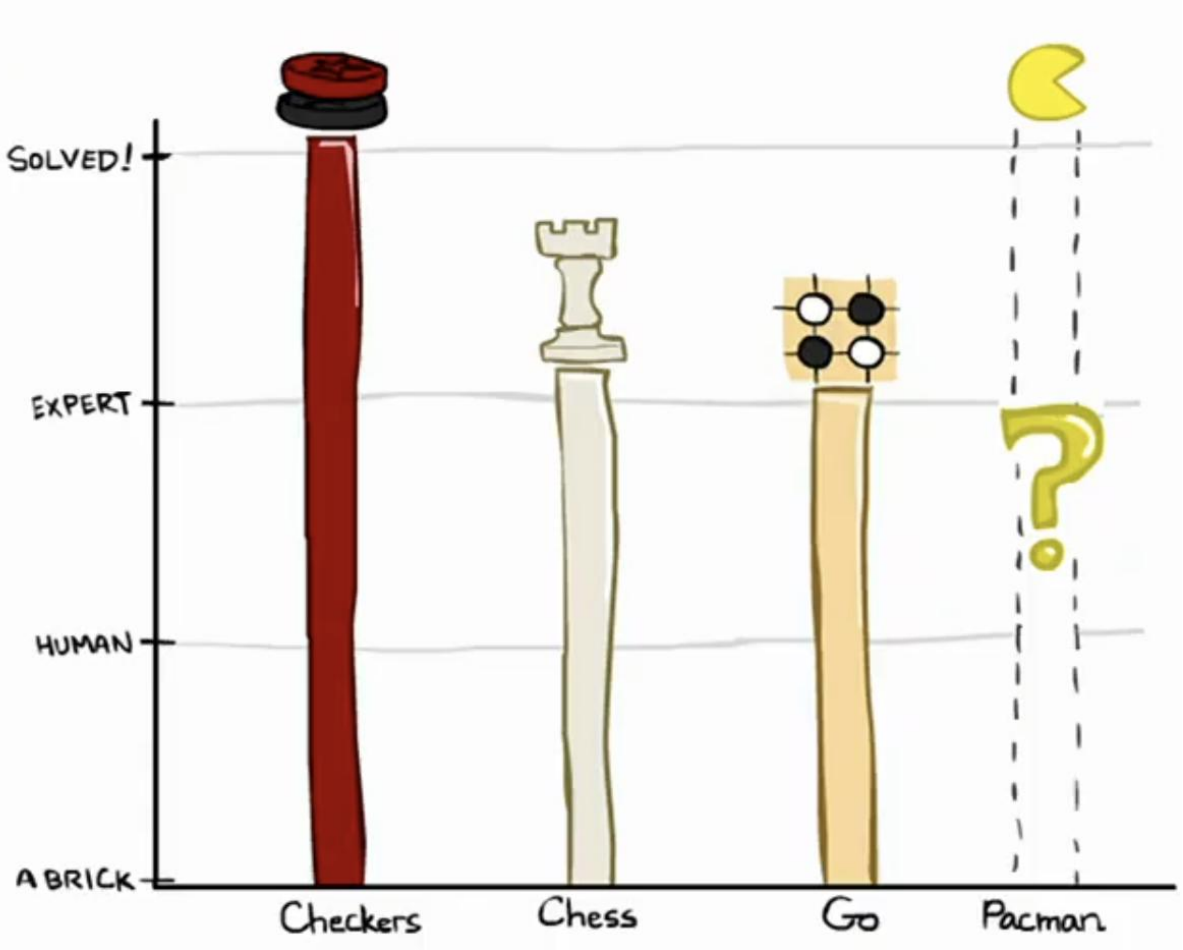
- CHESS:
  - 1997: **Deep Blue** beats Garry Kasparov in a six-game match. It looked at 200 million positions per second.
  - It used a strong evaluation function and extended some lines of search as deep as 40 ply.



Game Playing State-of-the-Art (2)

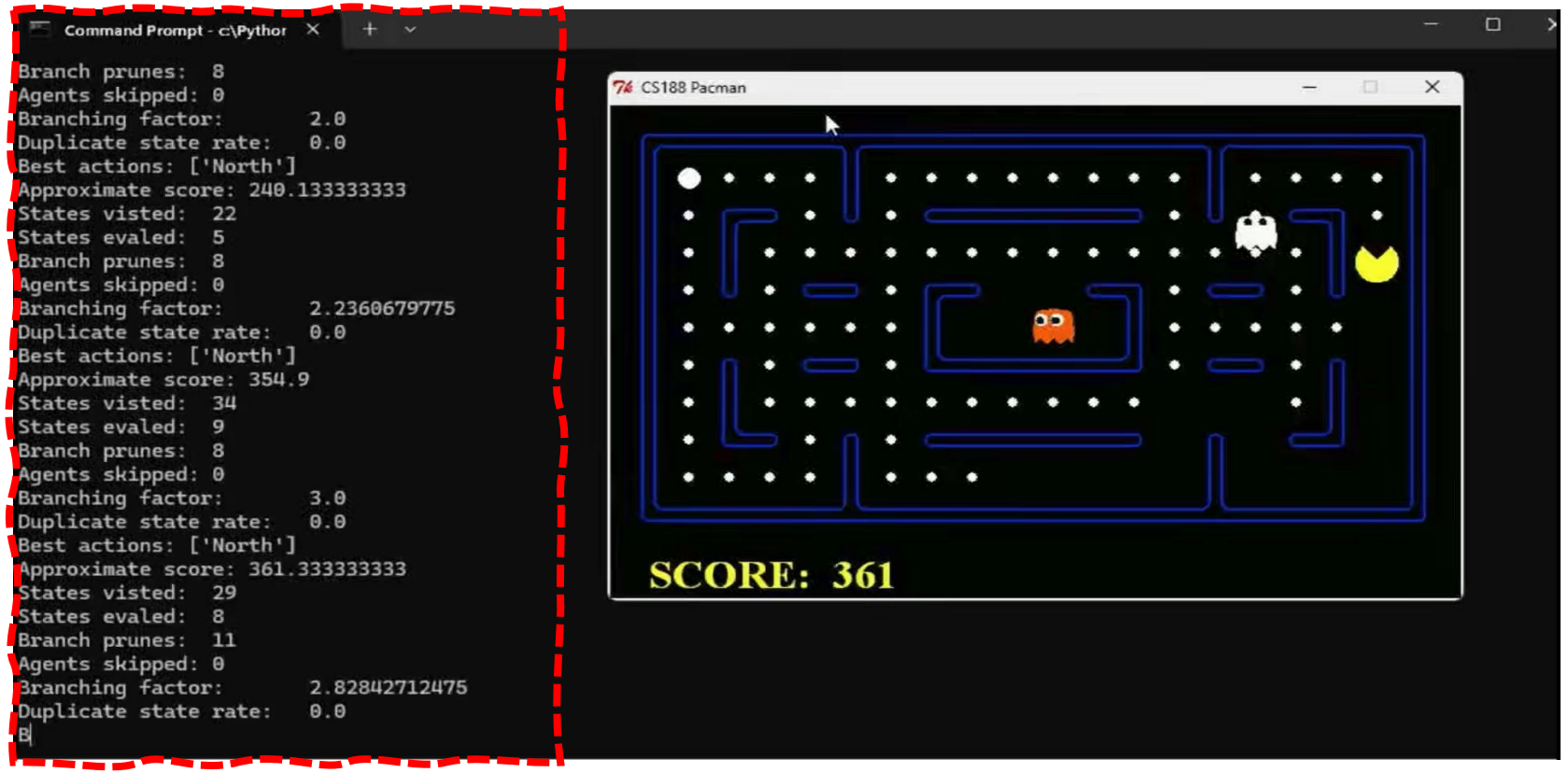
- GO: 2016, AlphaGo beats the human champion. It uses Monte Carlo Tree Search together with a learned evaluation function
  - Go took nineteen more years than chess. The board is larger, the branching factor is about 250, and nobody could write a good evaluation function by hand
  - That gap is the whole story of this chapter: searching is not enough. You also need a good way to judge a position you cannot finish

- And PAC-MAN: the running example for this course



Behavior from Computation

- A good choice for behavior: (1) Memory (learning; history) and (2) Computation (simulation; future)
- A Pac-Man demo. Watch the computation, not the game.
- Nobody wrote those behaviors. They come out of searching a tree
- One thing to notice: the scores on screen look like 1,442.47. No real score in this game can look like that. What are those numbers?



1. Adversarial Games

What changes when somebody else is trying to beat you

- The axes (dimensions; categories) along which games differ
  - A formal definition of **a deterministic game**
  - **Zero-sum games**, and why one number is enough
  - **Plans** vs **Policies**



Types of Games

- Many Different Kinds of Games

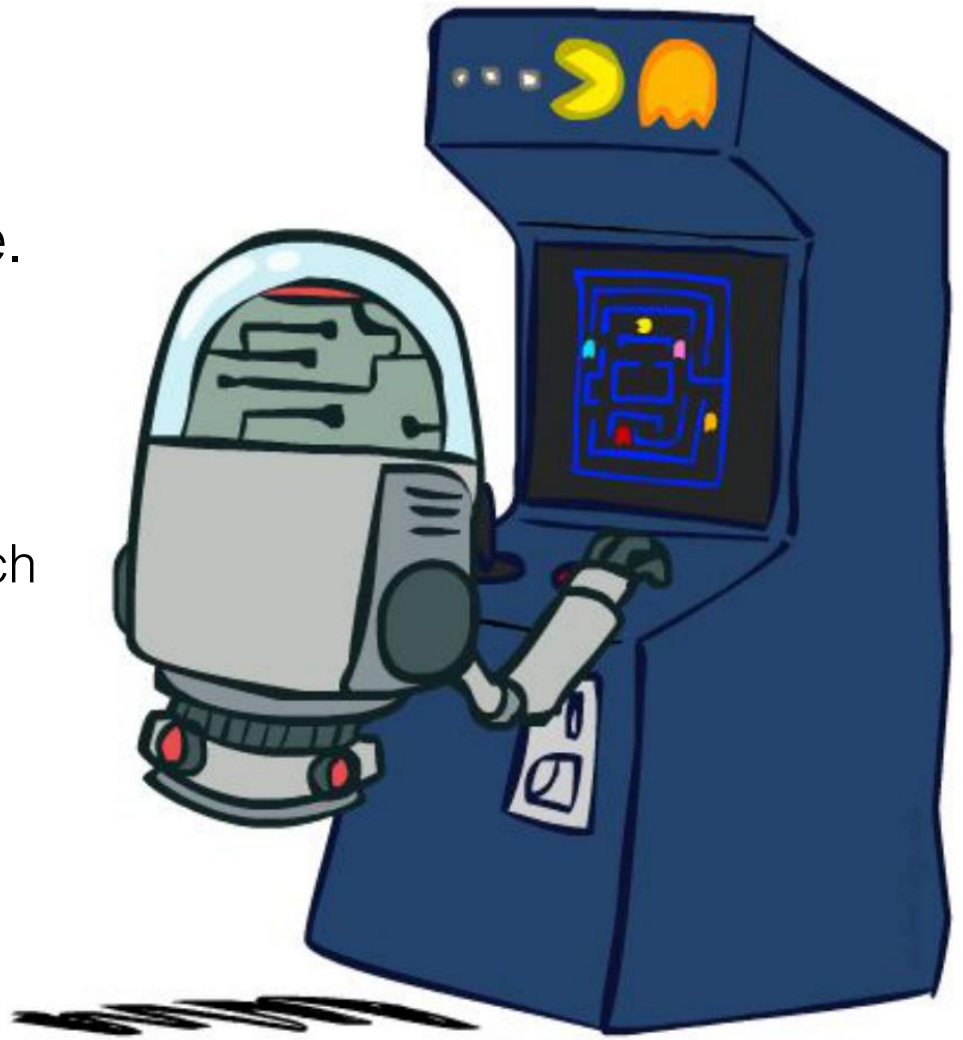


- Axes:
  1. **Deterministic or stochastic?**
     Chess is deterministic. Monopoly is not - you roll dice
  2. **One, two, or more players?**
     One player is just ordinary search
  3. **Zero sum?**
     If one side wins exactly what the other loses.
     Real life is usually not like this
  4. **Perfect information (can you see the state)?**
     Can you see the whole state, or is part of it hidden from you?

- We want algorithms that compute *a STRATEGY (a policy)* which recommends a move from each state, not a single plan

Deterministic Games

- Many possible formalizations, one is:
  - States: S, starting at s_0.
  - Players: $P = \{1 \ldots N\}$, usually taking turns
  - Actions: A (may depend on the player or the state.
  - Transition function: $S \times A \rightarrow S$
  - A terminal test : $S \rightarrow \{\text{True}, \text{False}\}$
  - Terminal Utilities: $S \times P \rightarrow R$
    - The utilities are the new part. In search, cost sat on each action. Here you play to the end and get one number.
- A solution is a *POLICY* $\pi: S \rightarrow A$



Zero-Sum Games



- Zero-Sum Games
  - The players have opposite utilities (values on outcomes). What one gains, the other loses.
  - Let's us think of a single value that one maximizes and the other minimizes.
  - Adversarial, pure competition.

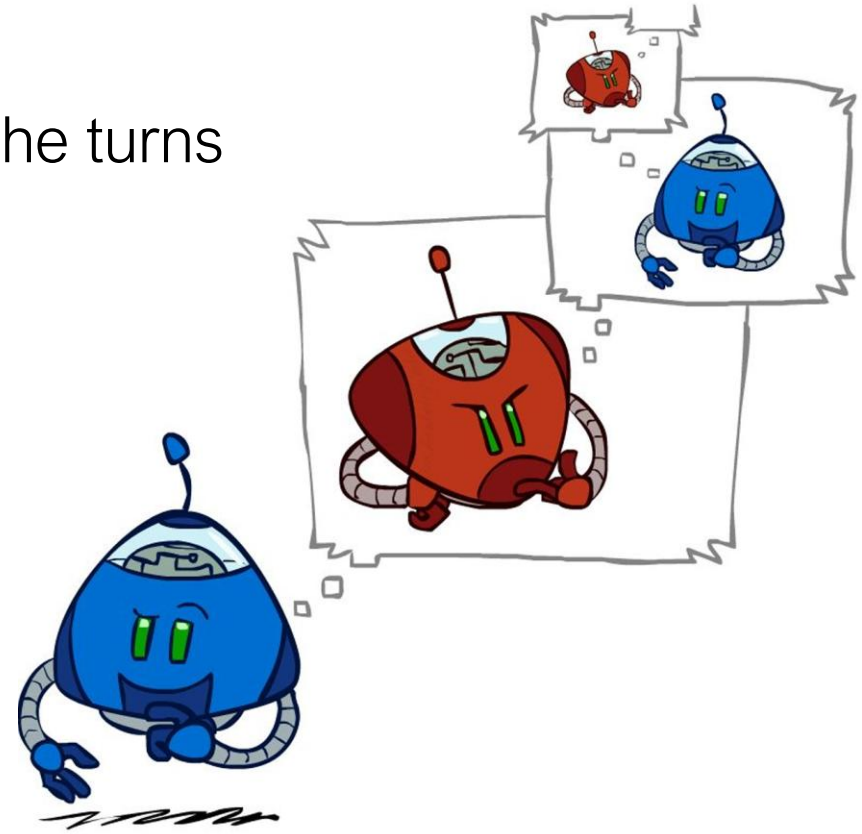


- General Games
  - Each player has an independent utility.
  - Cooperation, indifference and competition are all possible
  - Between the two extremes you get players who cooperate in some parts of the tree and compete in others

2. Adversarial Search

The value of a state, when somebody else controls half the moves

- Single-agent trees, and the value of a state
- What changes when an opponent takes some of the turns
- Minimax values, and the minimax algorithm
- How expensive it is, and why that is a problem



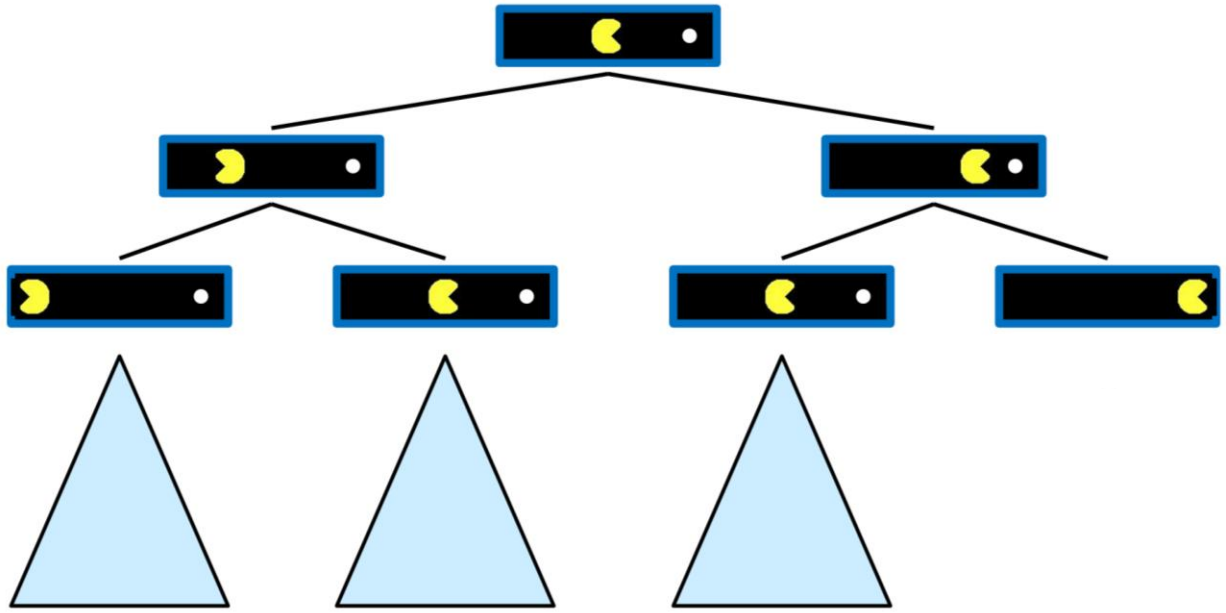
Single-Agent Trees

- Start with the easy case: only you are playing (***a single agent tree***)

- From the current state you have some actions. Each one leads to a new state, and from there more actions, and so on.

- This is exactly a search problem. You control every action, so **you can make any branch happen.**

- At the bottom, the numbers are utilities. A score of 8 here might be +10 for the dot and −1 for each of two steps.



Value of a State in Single-Agent Trees

- **Value of a state**: the best outcome (utility) you can reach from it
  → *the best achievable utility*

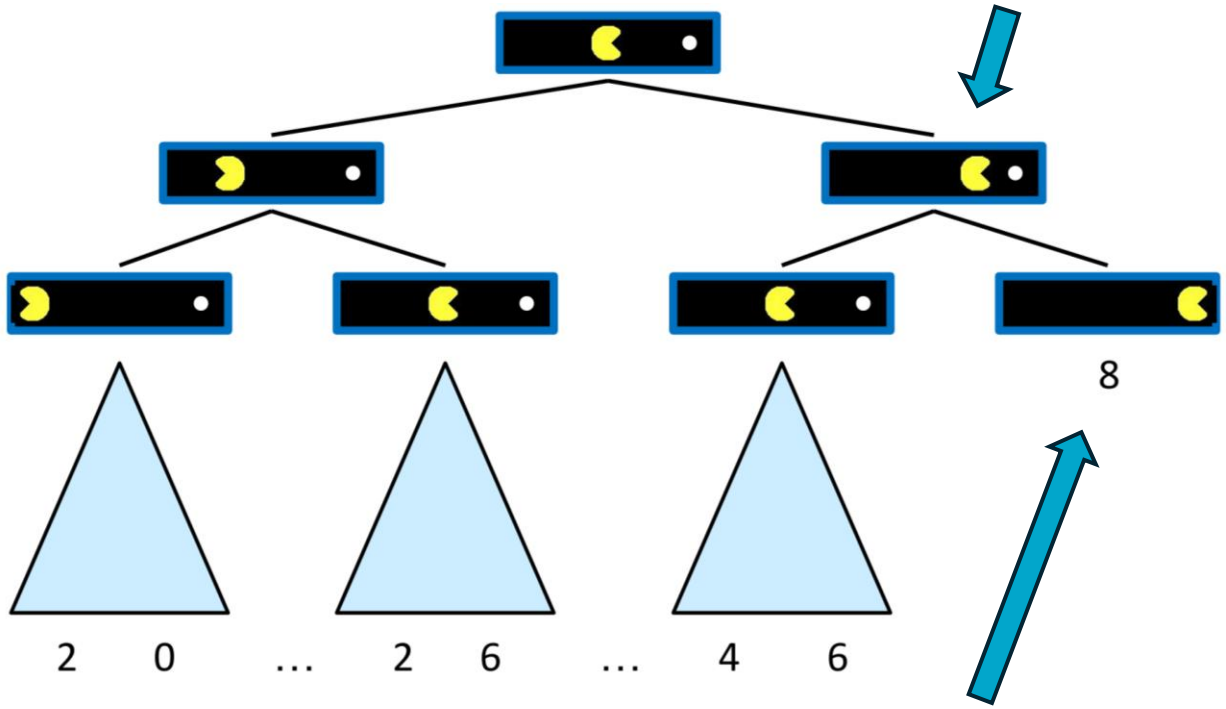
- (1) At a terminal state, the value is given. The game tells you.

- (2) At any other state, the value is the **MAXIMUM** of the values of its children.

- So, values flow upward from the bottom. The definition is ***recursive.***

(2) **Non-Terminal States:**

$$V(s) = \max_{s' \in \text{children}(s)} V(s')$$



(1) **Terminal States:**

$$V(s) = \text{known}$$

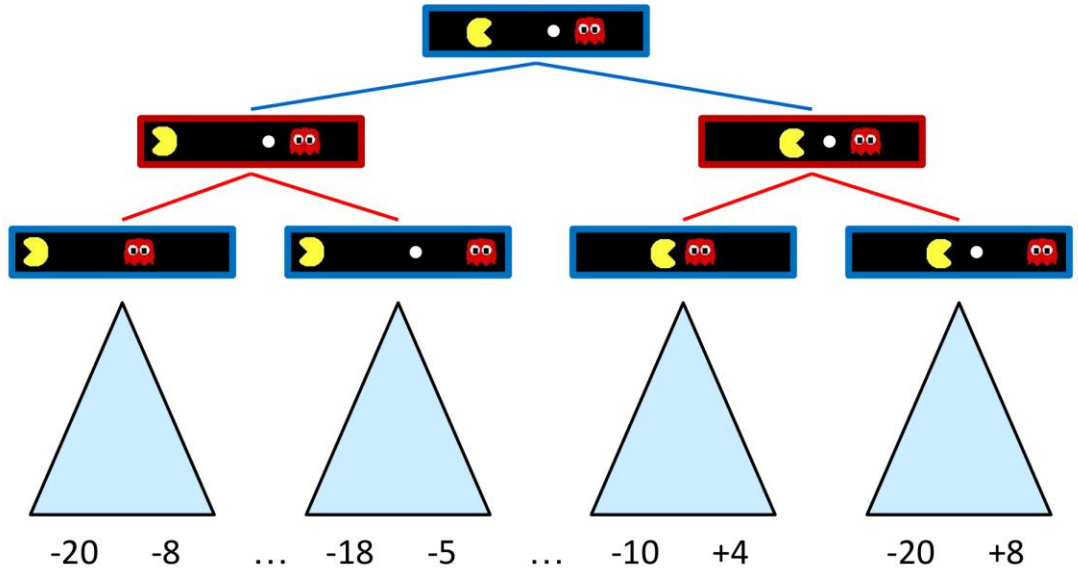
Adversarial Game Trees

- Now add an opponent. The tree holds the same set of futures as before

- The only thing that changed is <u>CONTROL</u>.
  - At some nodes you choose.
  - At others, the ghost chooses

- You do not know what the ghost will do. But you do know what it <u>CAN</u> do

- And in a zero-sum game you know what it WANTS: the opposite of what you want



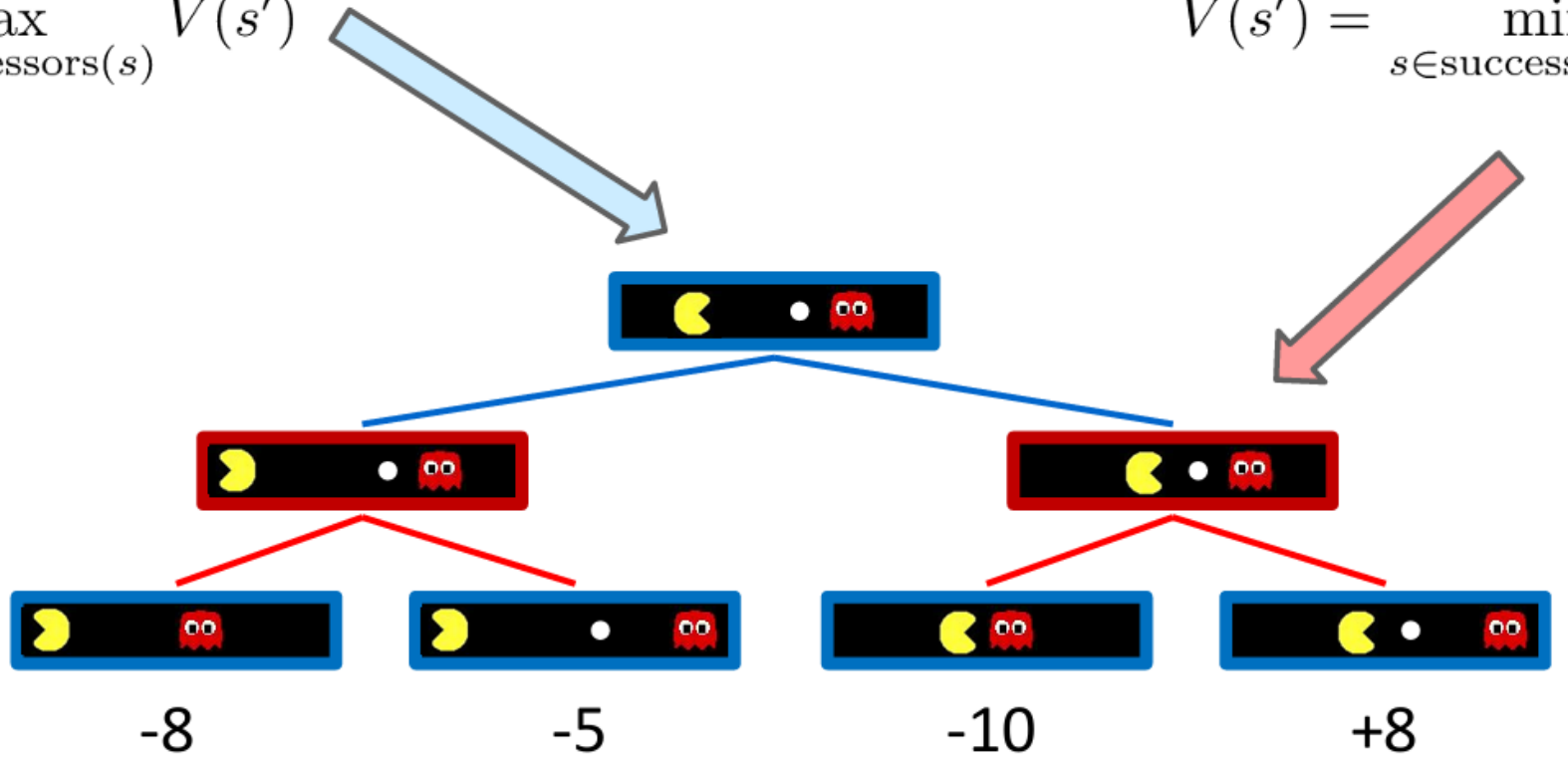
Minimax Values in Adversarial Game Trees

States Under Agent's Control:

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

States Under Opponent's Control:

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$

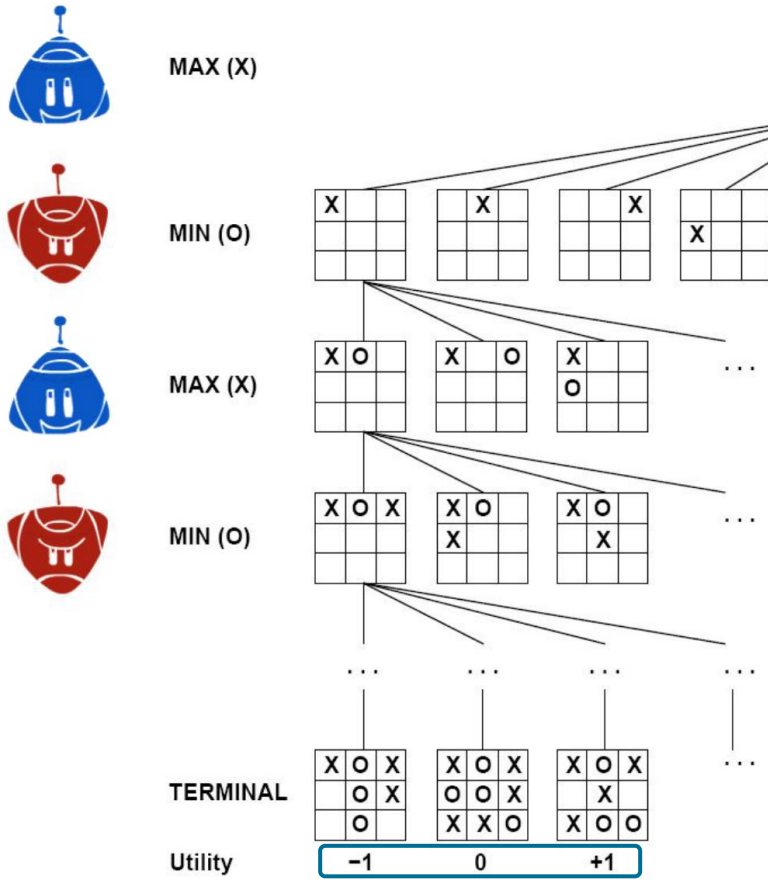


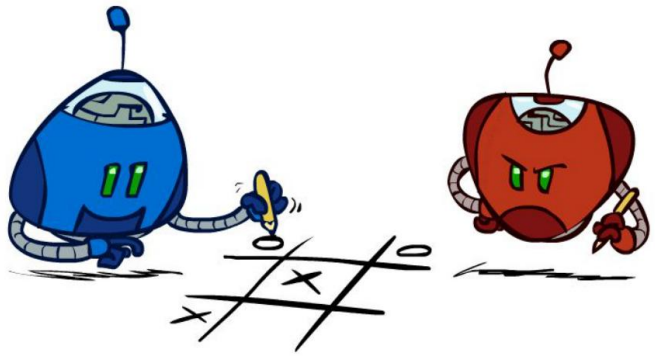
Terminal States:

$$V(s) = \text{known}$$

Example: Tic-Tac-Toe Game Tree

- The same idea on a game everybody knows.
- In tic-tac-toe, the terminal utilities are only +1, 0 and −1: I win, we draw, I lose.
- There is no averaging anywhere here. It is a $\max(\min(\max(\min(\max(\ldots)))))$
  → **The root's value must be one of the leaf values.**

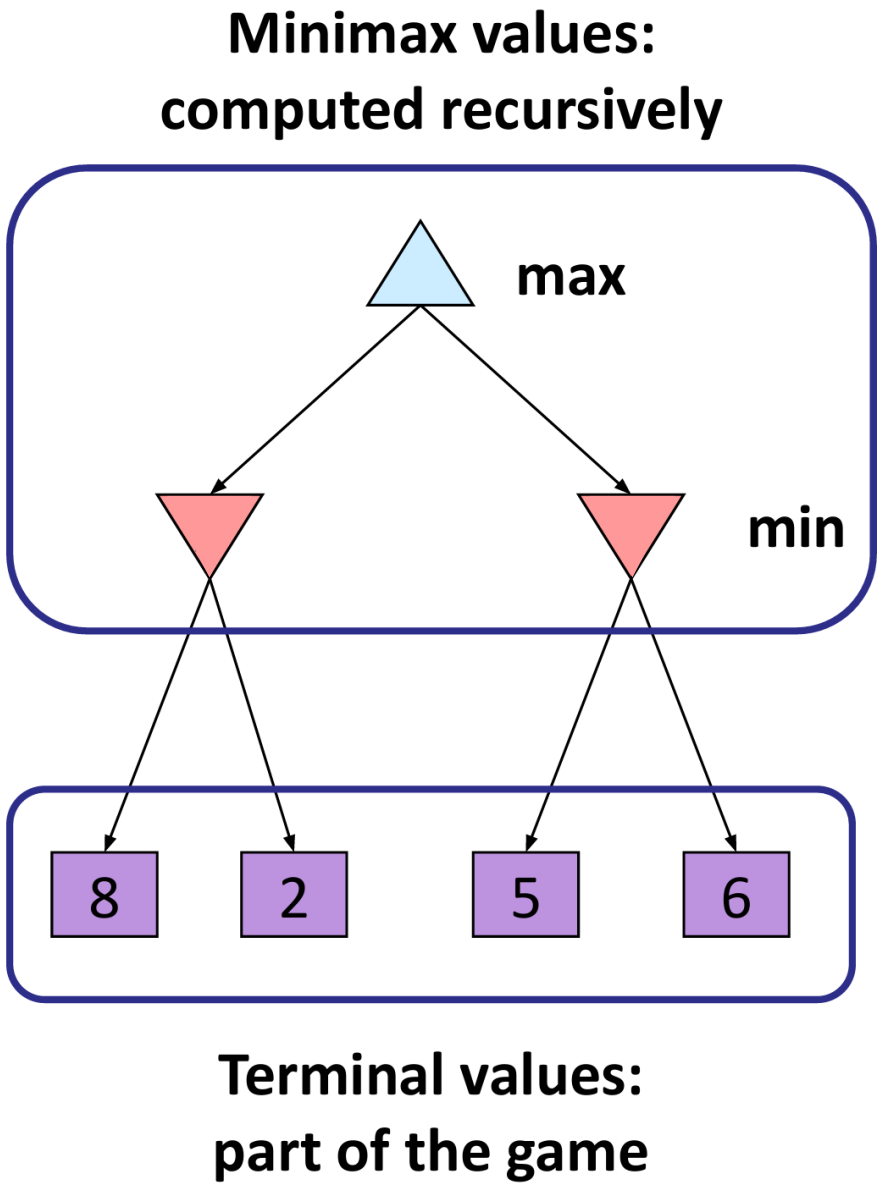




Adversarial Search (Minimax)

- The setting: deterministic, zero-sum games
  - Tic-tac-toe, chess and checkers
  - One player maximizes the result.
  - The other minimizes it

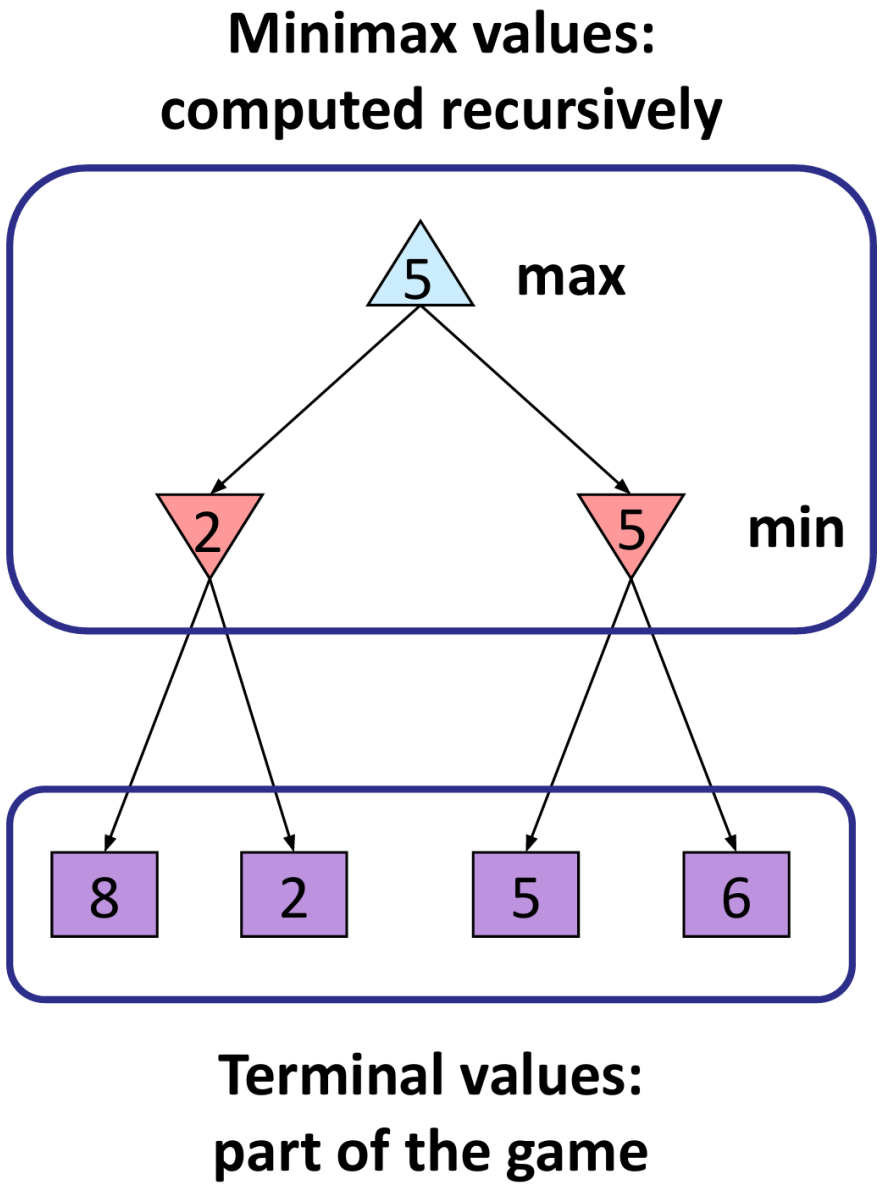
- Minimax search:
  - A state-space search tree
  - Players alternate turns.
  - Compute each node's minimax value: the best achievable utility against a rational (optimal) opponent.



Adversarial Search (Minimax)

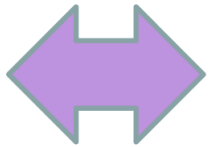
- The setting: deterministic, zero-sum games
  - Tic-tac-toe, chess and checkers
  - One player maximizes the result.
  - The other minimizes it

- Minimax search:
  - A state-space search tree
  - Players alternate turns.
  - Compute each node's minimax value: the best achievable utility against a rational (optimal) opponent.



Minimax Implementation

```
def max-value(state):
    initialize v = -∞
    for each successor of state:
        v = max(v, min-value(successor))
    return v
```



```
def min-value(state):
    initialize v = +∞
    for each successor of state:
        v = min(v, max-value(successor))
    return v
```

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$

- Two functions that call each other: a mutual recursion.
- Watch the bounds move. In max-value, v starts at -inf and **RISES**. In min-value, v starts at +inf and **FALLS**. Alpha-beta needs this.

Minimax Implementation (Dispatch)

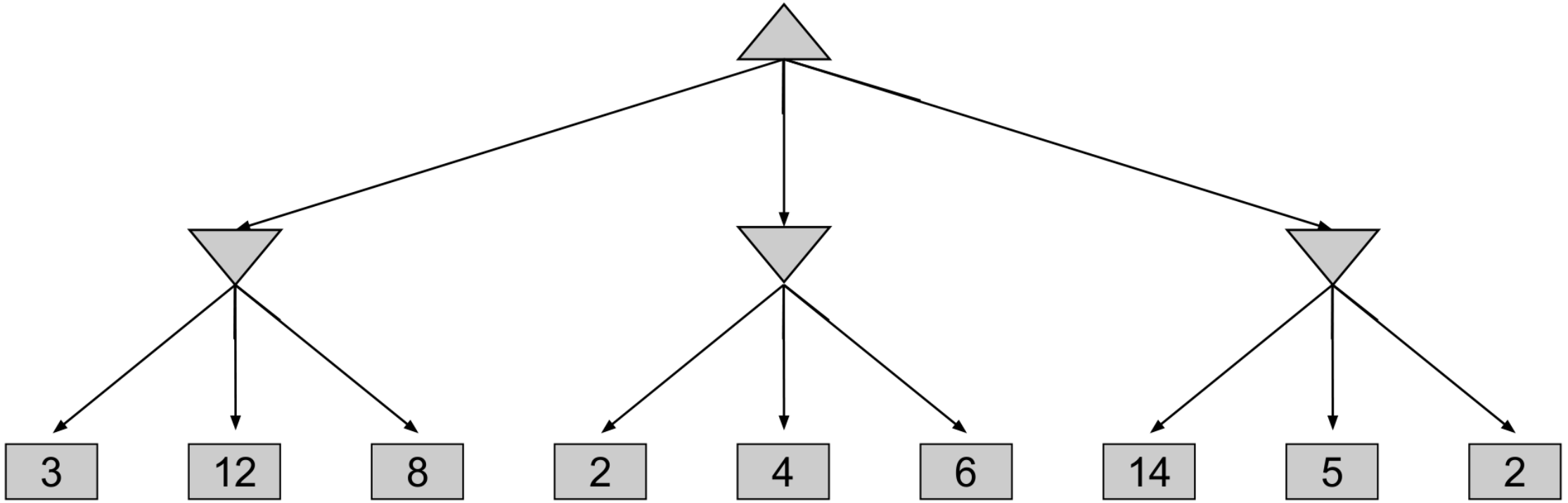
```
def value(state):
    if the state is a terminal state: return the state's utility
    if the next agent is MAX: return max-value(state)
    if the next agent is MIN: return min-value(state)
```

```
def max-value(state):
    initialize v = -∞
    for each successor of state:
        v = max(v, value(successor))
    return v
```

```
def min-value(state):
    initialize v = +∞
    for each successor of state:
        v = min(v, value(successor))
    return v
```

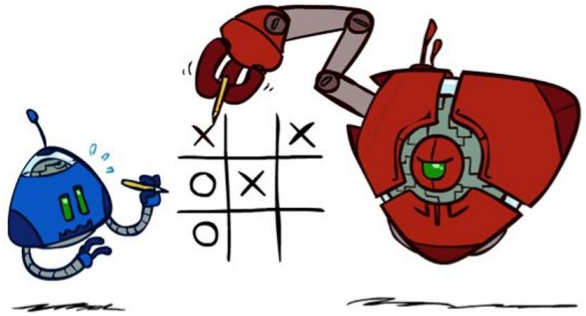
*Every state has a **value**. What decides it is who controls the state.*

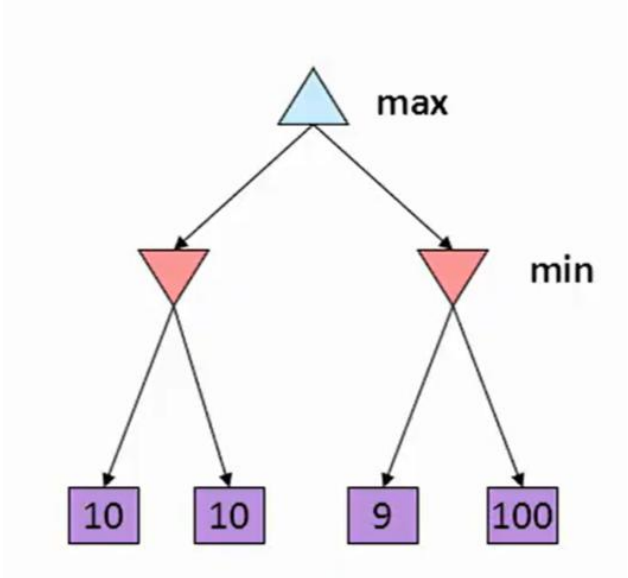
Minimax Example

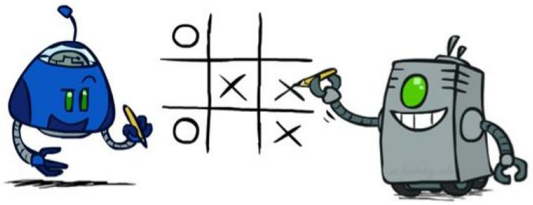


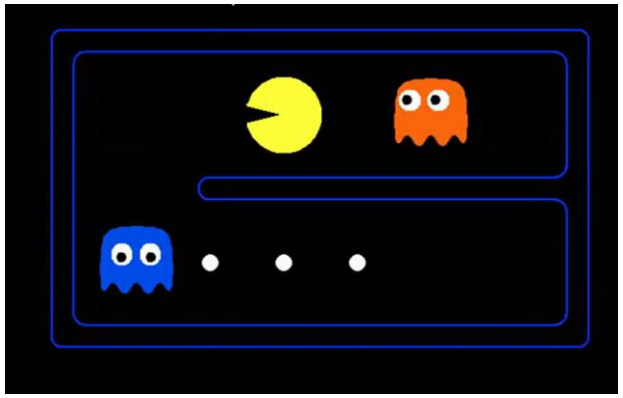
Minimax Properties

- Minimax is optimal against a perfect player. That is the guarantee
  - What if the opponent is not perfect? Then you still do at least as well as the minimax value. Their mistakes can only help you
  - But you may do much worse than you could have 미내. Minimax will not take a risk that a perfect opponent would punish, even against an opponent who would not punish it
- In the demo: against a ghost that moves randomly, minimax Pac-Man throws itself at the ghost









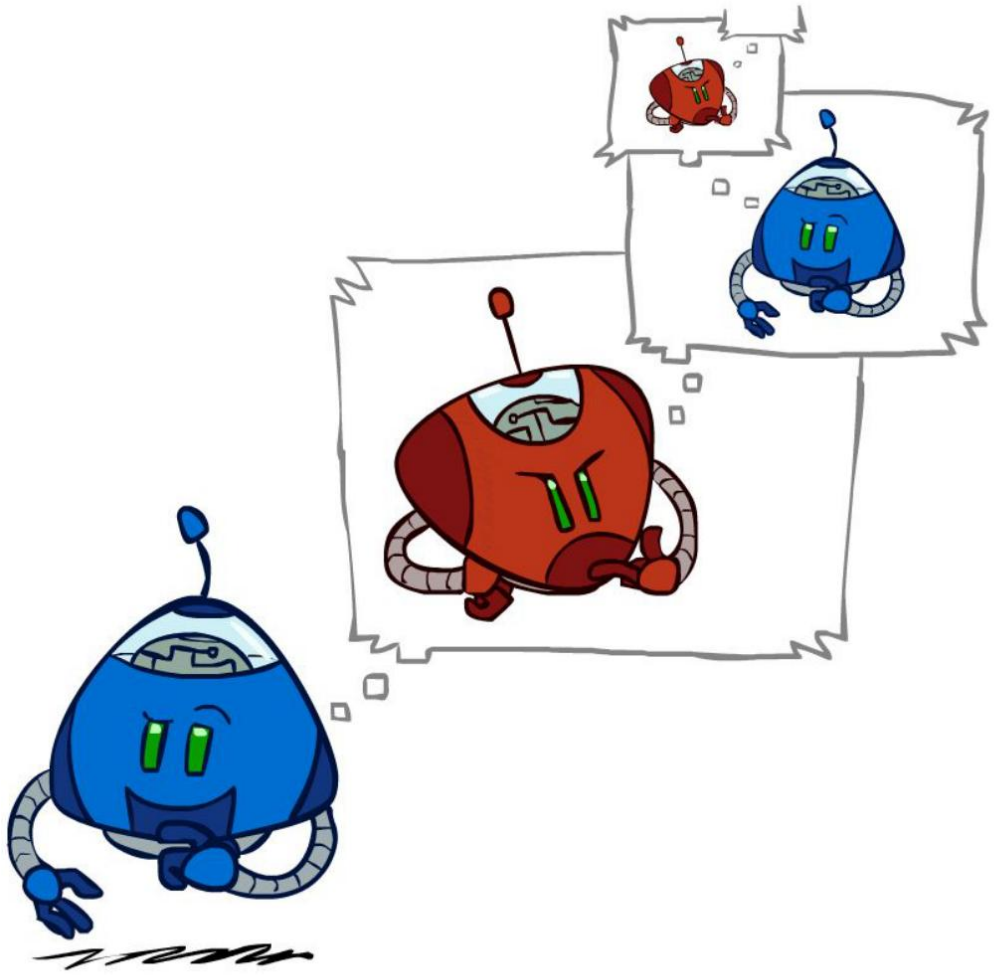
Optimal against a perfect player. Otherwise?

Minimax Efficiency

- How efficient is minimax?
  - Just like (exhaustive) DFS
  - Time: $O(b^m)$
  - Space: $O(bm)$

- Unlike ordinary search we cannot stop at the first goal. We have to look at the WHOLE tree.
  - Why? Because you never know whether the part you skipped is the part that decides the game

- For chess, b ≈ 35 and m ≈ 100.
  - An exact solution is completely impossible: $O(35^{100})$



Introduction to Artificial Intelligence

CS188 6 - Part II: Game Tree Pruning

Namyup Kim

Soongsil University

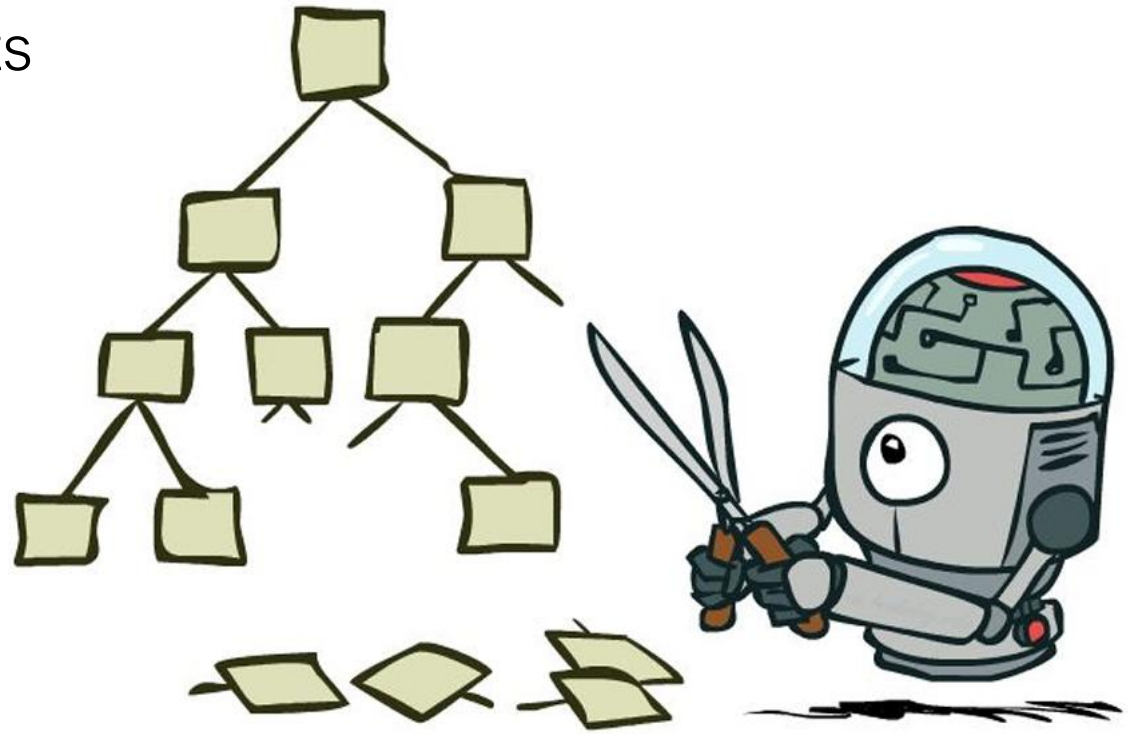


Source: UC Berkeley CS188, slides by Dan Klein and Pieter Abbeel

3. Game Tree Pruning

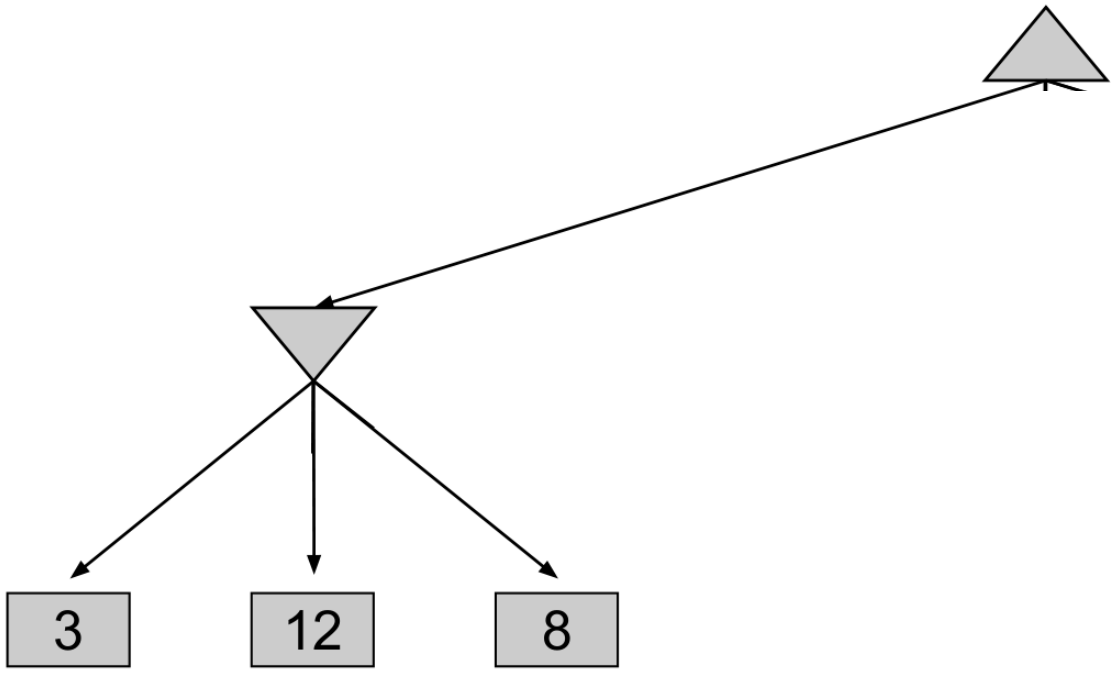
Getting the same answer without looking at the whole tree

- Working the example again, watching the bounds
- The general rule, and the two numbers **alpha** and **beta**
- What pruning changes, and one trap it sets
- Two quizzes



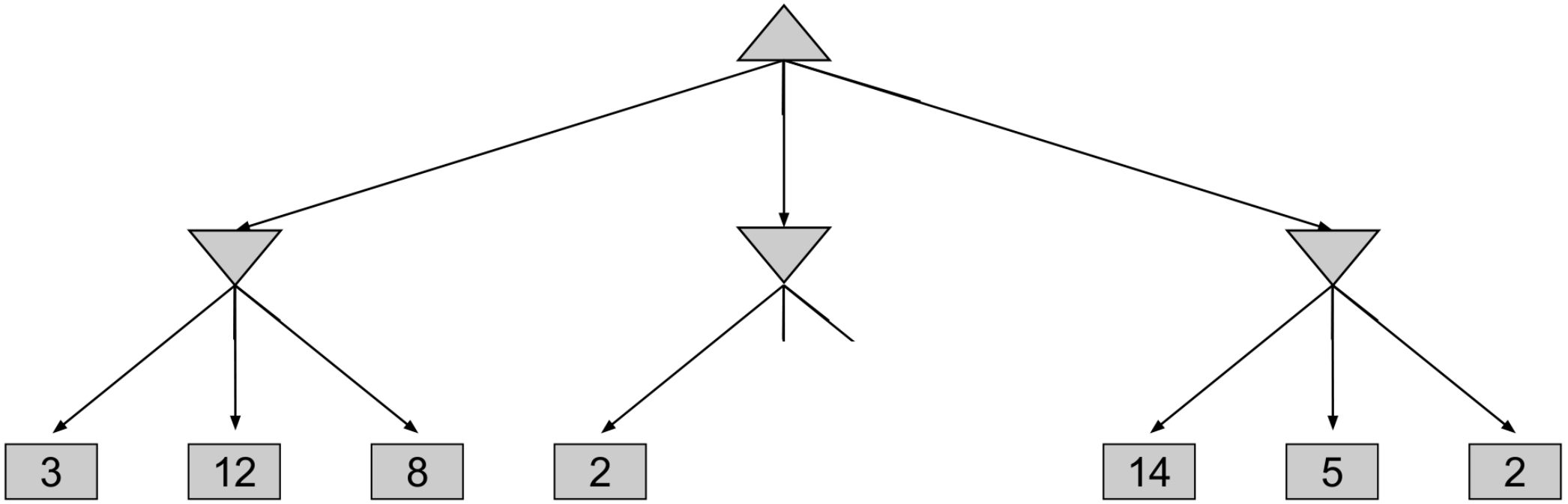
Minimax Example

- The first MIN node sees 3, then 12, then 8, and settles on 3.
- The root now knows it can get at least 3. That is a lower bound, and it will never go down.
- Keep that number in mind as we move to the second branch.



Minimax Pruning

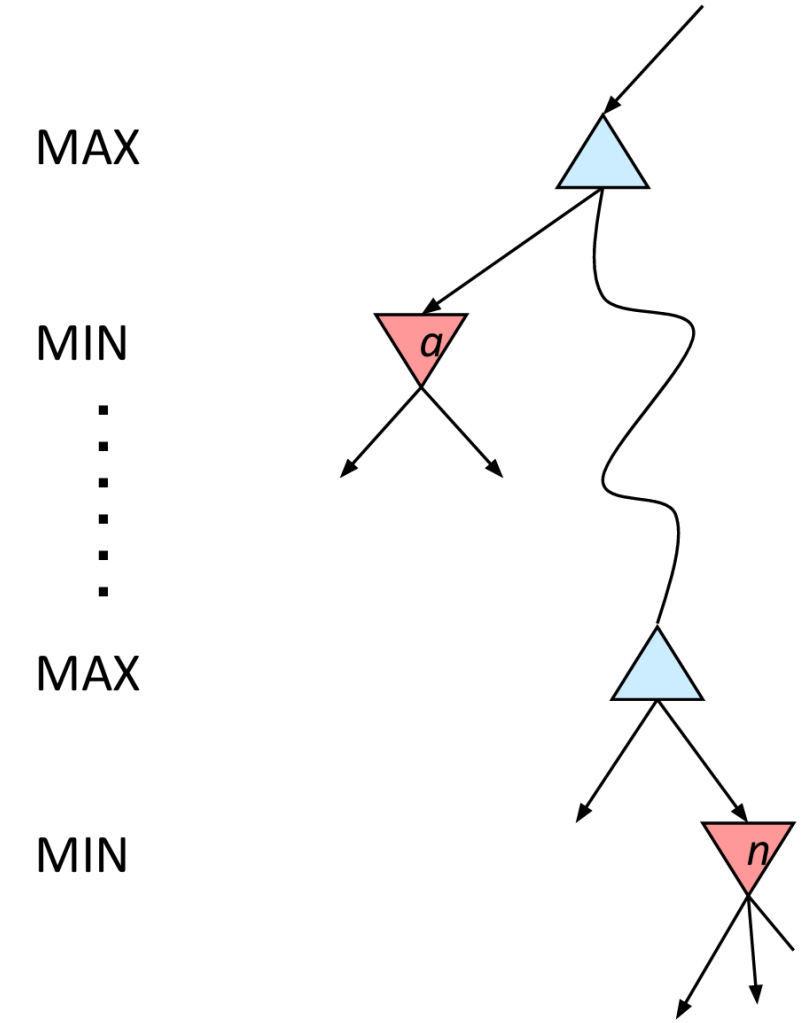
- Move to the second branch. Its first child is 2, so this MIN node is at most 2
- The root already has 3. A branch worth at most 2 can never beat it
- So, we stop. We do not look at the rest of that branch at all
- Notice what we gave up: we never learn the real value of that node. We only learn that it does not matter



Alpha-Beta Pruning

- General configuration (MIN version)
  - We're computing the MIN-VALUE at some node n
  - We're looping over n's children
  - n's estimate of the children's min is dropping its value
  - Who cares about n's value? MAX
  - Let ALPHA α be the best value that MAX can get *at any choice point on the path to the root*
  - As soon as v drops below alpha, this node can never be chosen. Stop and return

- BETA β is symmetric



Alpha-Beta Implementation

α: MAX's best option on path to root
β: MIN's best option on path to root

```
def max-value(state, α, β):
    initialize v = -∞
    for each successor of state:
        v = max(v, value(successor, α, β))
        if v ≥ β return v
        α = max(α, v)
    return v
```

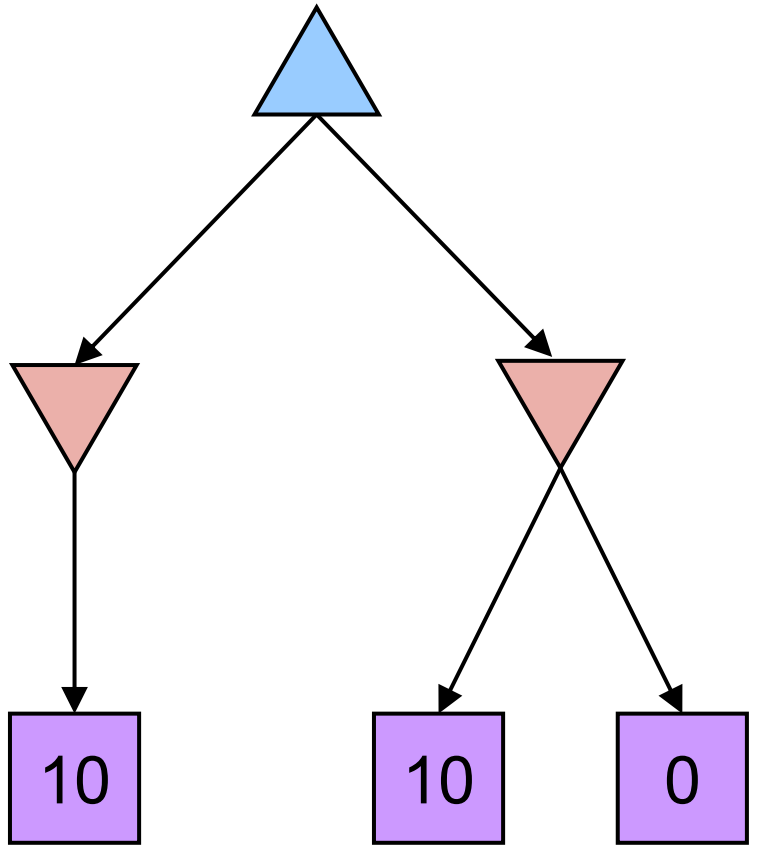
```
def min-value(state , α, β):
    initialize v = +∞
    for each successor of state:
        v = min(v, value(successor, α, β))
        if v ≤ α return v
        β = min(β, v)
    return v
```

Early stop, Prune!

Update the best MIN

Alpha-Beta Pruning Properties

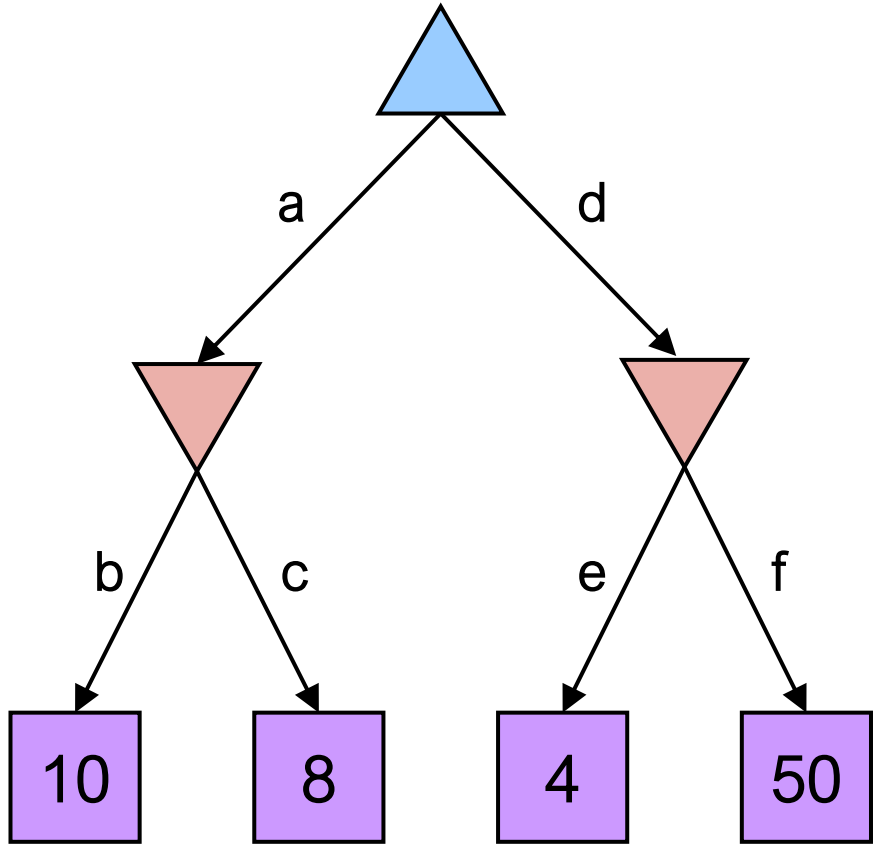
- Pruning has NO effect on the minimax value computed for the root. The answer is exactly the same!
- But the values of intermediate nodes may be **WRONG**, because we stopped early and returned a bound instead of a value.
- So, the naive version does not let you do the best *Action Selection* off the root. You need correctness one level down.
- Good child ordering matters enormously. With perfect ordering the time goes from $O(b^m)$ to $O(b^{\frac{m}{2}})$
- This is a simple example of metareasoning (computing about what to compute)



Alpha-Beta Quiz 1

- Work out the minimax value of the root

- Then: visiting left to right, which branches can alpha-beta prune?



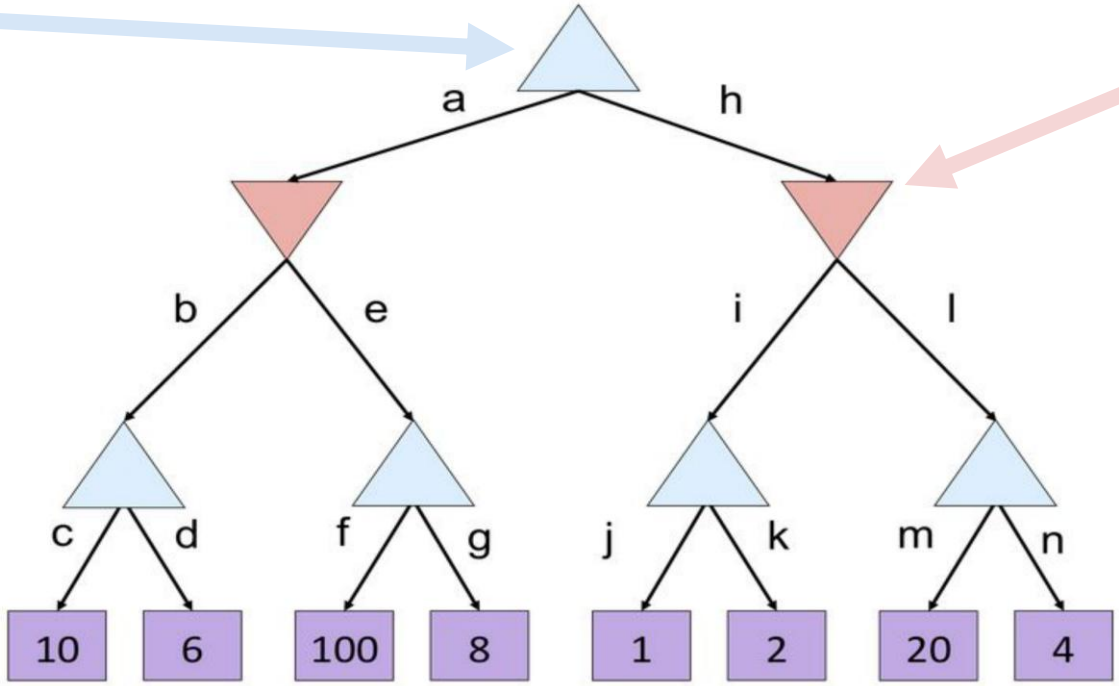
Alpha-Beta Quiz 2

- This one is harder to do in your head, so write the bounds down.
- Watch for the configuration where you already know the value of an earlier branch - that is where the pruning comes from

α: MAX's best option on path to root
β: MIN's best option on path to root

```
def max-value(state, α, β):
    initialize v = -∞
    for each successor of state:
        v = max(v, value(successor, α, β))
        if v ≥ β return v
        α = max(α, v)
    return v
```

```
def min-value(state , α, β):
    initialize v = +∞
    for each successor of state:
        v = min(v, value(successor, α, β))
        if v ≤ α return v
        β = min(β, v)
    return v
```



4. Resource Limits

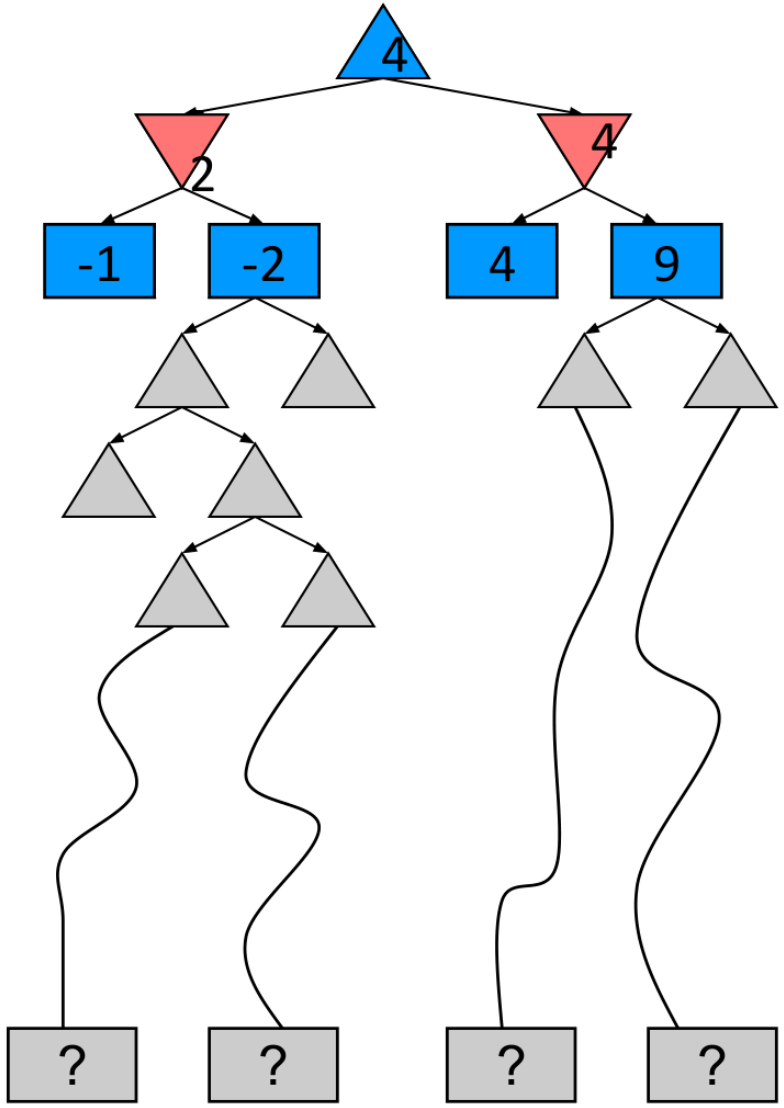
When you cannot reach the leaves, you have to invent them

- Depth-limited search, and why it needs a base case
- Replacing terminal utilities with an evaluation function
- Why a replanning Pac-Man can starve next to a dot

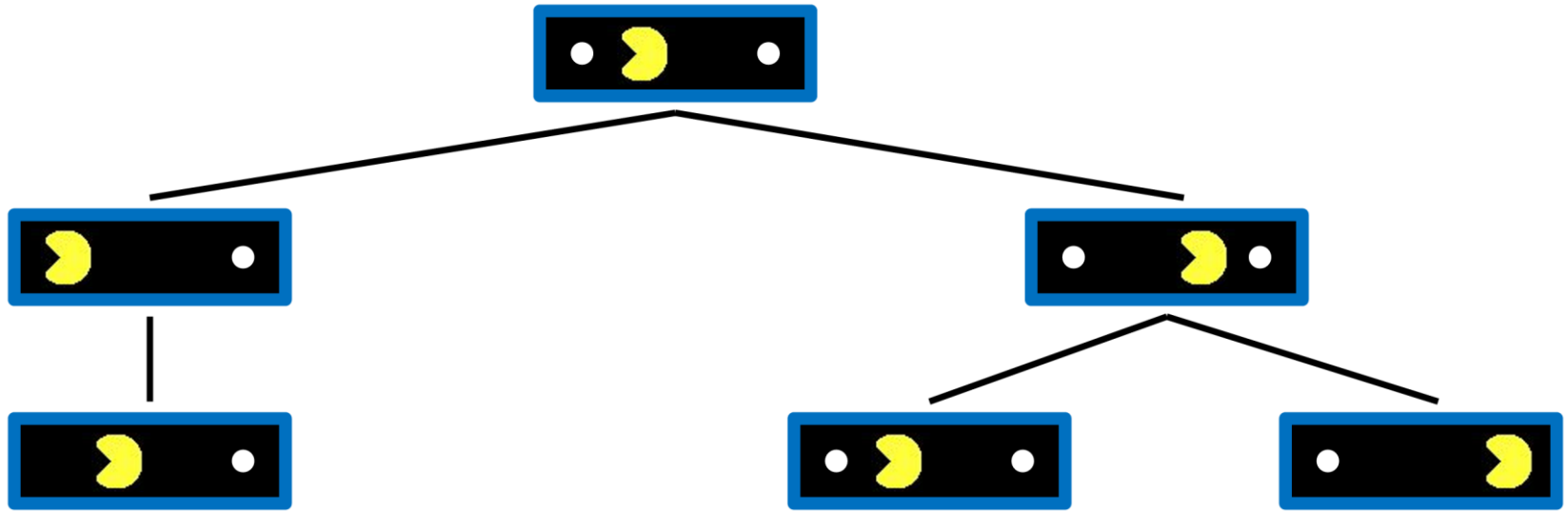


Resource Limits

- Problem: In realistic games, cannot search to leaves!
- Solution: Depth-limited search
  - Instead, search only to a limited depth in the tree
  - Replace terminal utilities with an evaluation function for non-terminal positions
- Example:
  - Suppose we have 100 seconds, can explore 10K nodes / sec
  - So can check 1M nodes per move
  - α-β reaches about depth 8 – decent chess program
- Not guarantee of optimal play
- More plies makes a BIG difference
- Use iterative deepening for an anytime algorithm



Why Pacman Starves

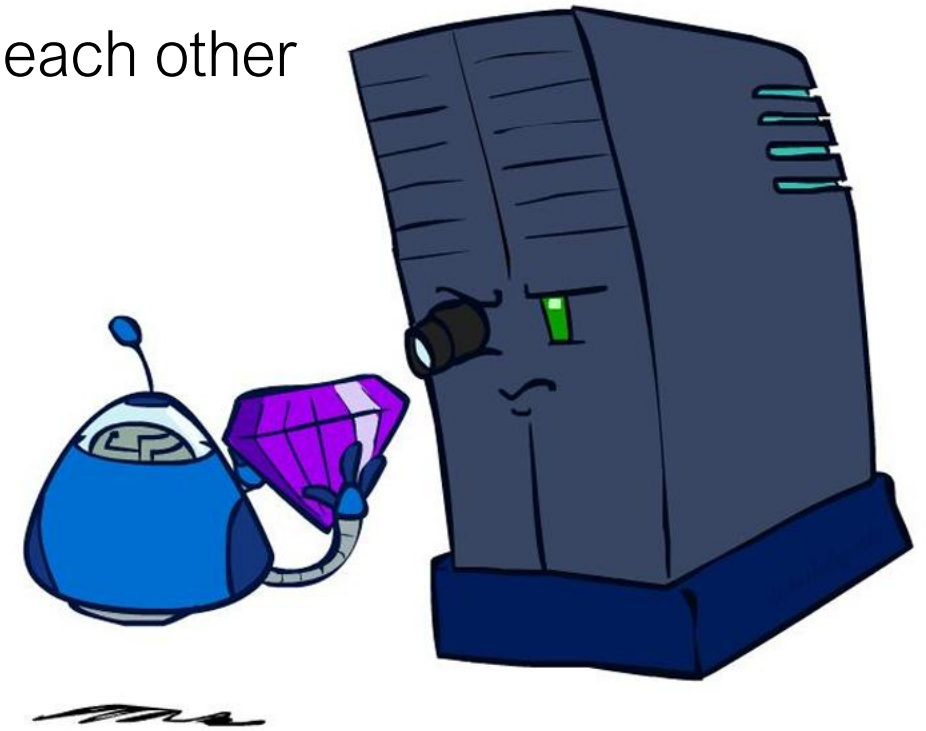


- A danger of “replanning” agents!
  - Pac-Man knows his score goes up if he eats the dot now – (west, then east)
  - He also knows it goes up by exactly the same amount if he eats it later – (east, then west)
  - Within his horizon, there are no other point-scoring chances after the dot
  - Therefore, waiting seems just as good as eating: he may go east, then back west in the next round of replanning!

5. Evaluation Functions

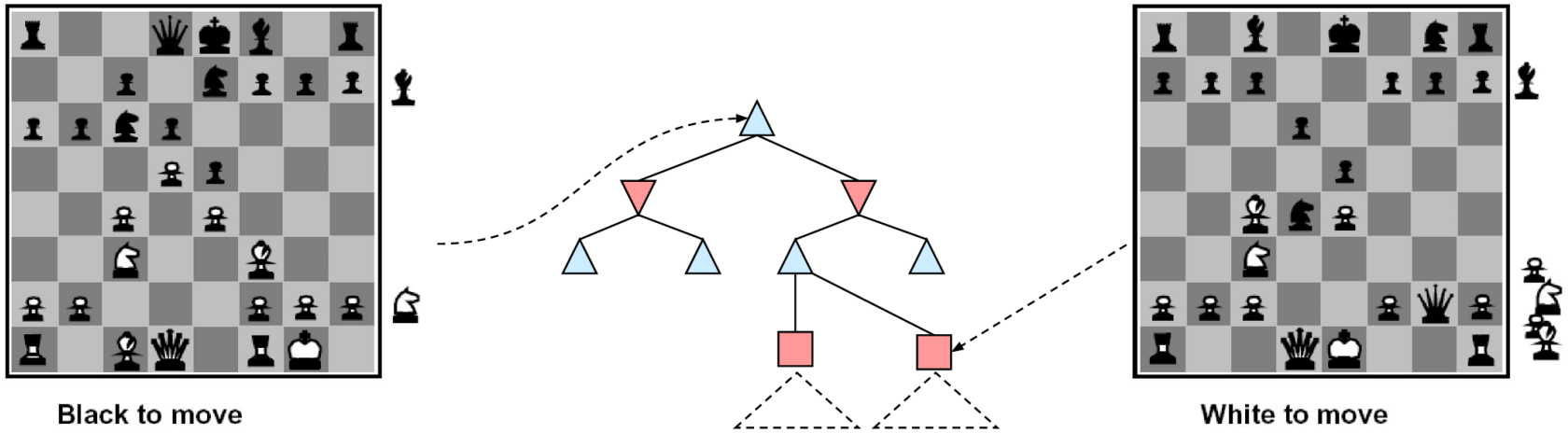
The number you put in when you stop early

- What an evaluation function is trying to be
- The usual form: a weighted sum of features
- Why depth and evaluation quality trade off against each other
- How ordering connects pruning and evaluation



Evaluation Functions

- Evaluation functions score *non-terminals* in depth-limited search



- Ideal function: returns the actual minimax value of the position
- In practice: approximate, e.g. weighted linear sum of features:

$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \ldots + w_n f_n(s)$$

- e.g. f1(s) = (num white queens – num black queens), etc.

Introduction to Artificial Intelligence

Chapter 6 - Part III: Uncertainty and Utilities

Namyup Kim

Soongsil University

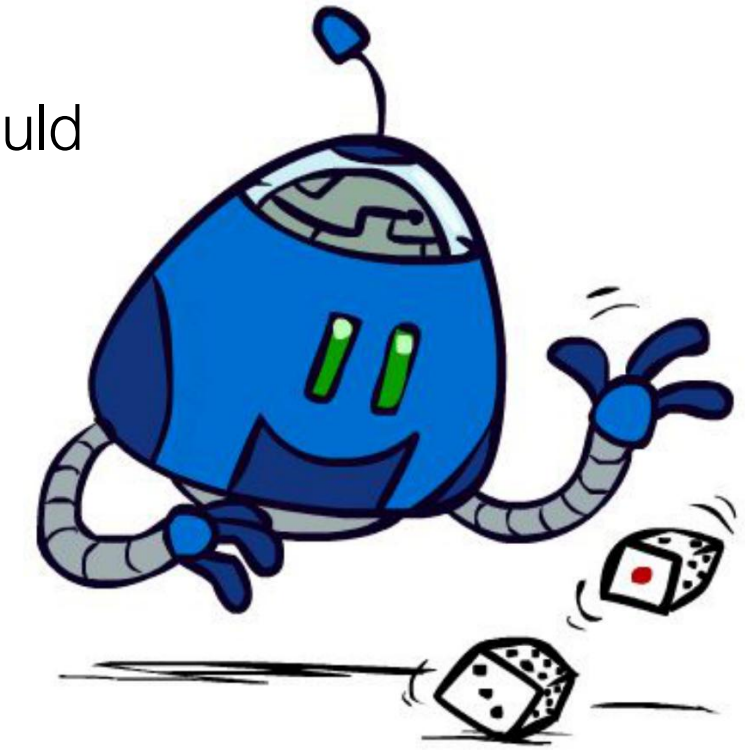


Source: UC Berkeley CS188, slides by Dan Klein and Pieter Abbeel (ai.berkeley.edu)

1. Uncertain Outcomes

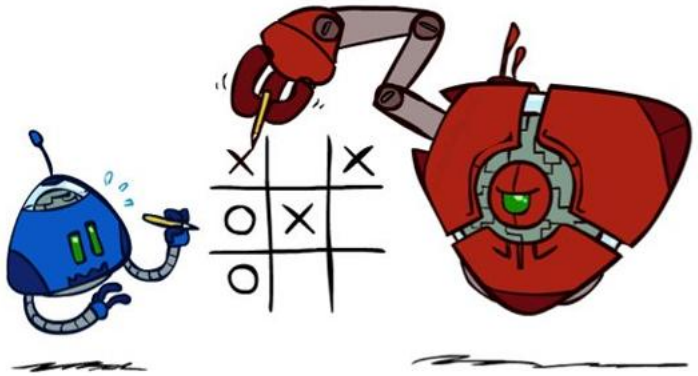
What happens when the other player is not trying to beat you

- **Worst case (MIN case)** vs **average case**, on the same tree
- **Expectimax**: the algorithm you get when **MIN** becomes an **average, not a selection.**
- Why expectimax **cannot be pruned** the way minimax could
- Depth limits again, now estimating an average

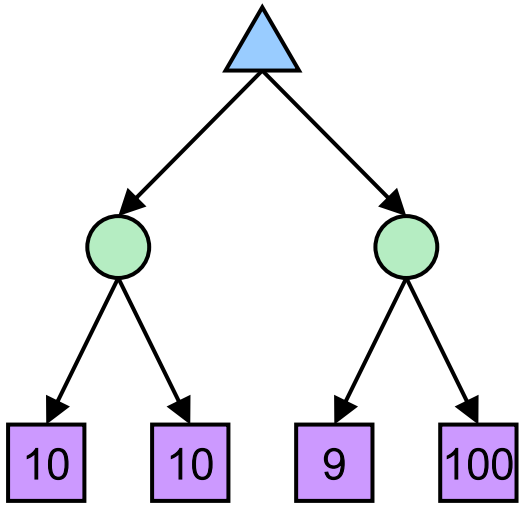


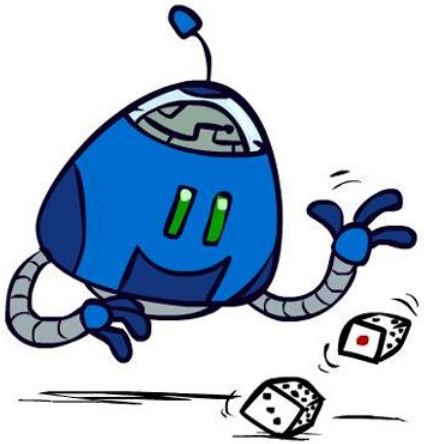
Worst-Case vs. Average Case

- The same tree, read two ways.
  - Against a minimizer (**mastermind**): left gives 10, right gives 9. The 100 is not yours.
  - Against a chance node ◯ (**rolling dice; averaging**): right is (9 + 100) / 2 = 54.5, so right wins.
- Same numbers, opposite answer. What changed is your model of that layer
- Is that leaf 100, or a billion? Under min and max it made no difference



Mastermind



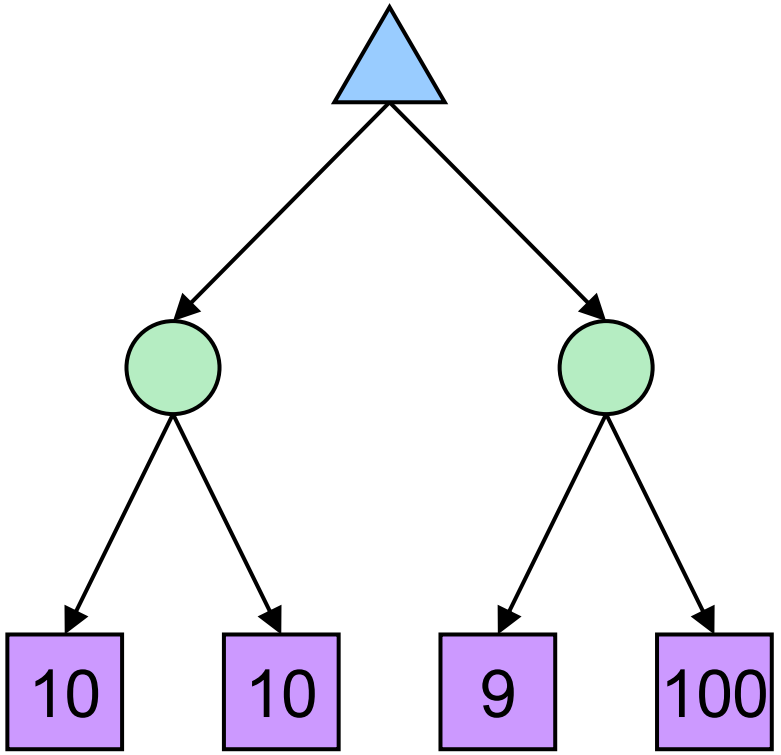


Rolling dice

Idea: Uncertain outcomes controlled by chance, not by an adversary!

Expectimax Search

- Why would we not know what an action does? Three sources:
  - Explicit randomness: rolling dice in Monopoly
  - Unpredictable opponents: random ghosts in PacMan
  - Actions failure: when moving a robot, a wheel slips.
- So, values should reflect the **average case (expectimax)** , not the worst case (minimax)
- ***Expectimax search***: compute the average score under optimal play
  - Max nodes as in minimax search
  - Chance nodes are like min nodes but the outcome is uncertain
  - Calculate their expected utilities
    - I.e., take weighted average (expectation) of children
- Later, we'll learn how to formalize the underlying uncertain-result problems as Markov Decision Processes, which is Chapter 7.



Expectimax Pseudocode

```
def value(state):
    if the state is a terminal state: return the state's utility
    if the next agent is MAX: return max-value(state)
    if the next agent is EXP: return exp-value(state)
```

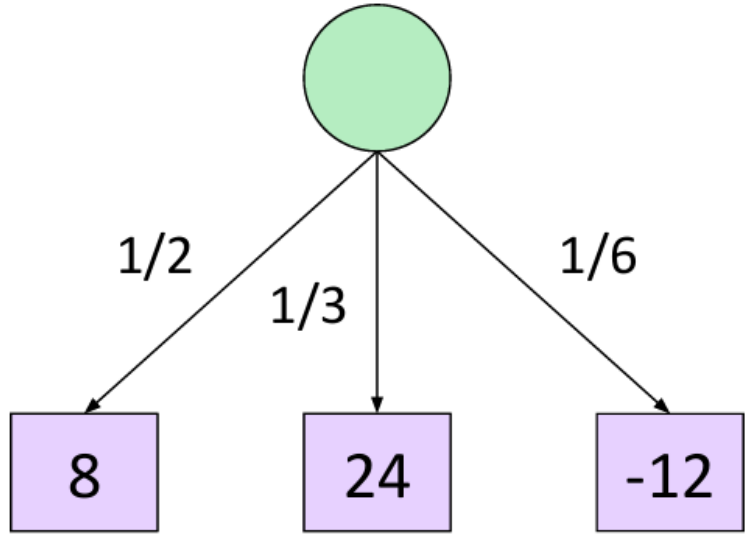
```
def max-value(state):
    initialize v = -∞
    for each successor of state:
        v = max(v, value(successor))
    return v
```

```
def exp-value(state):
    initialize v = 0
    for each successor of state:
        p = probability(successor)
        v += p * value(successor)
    return v
```

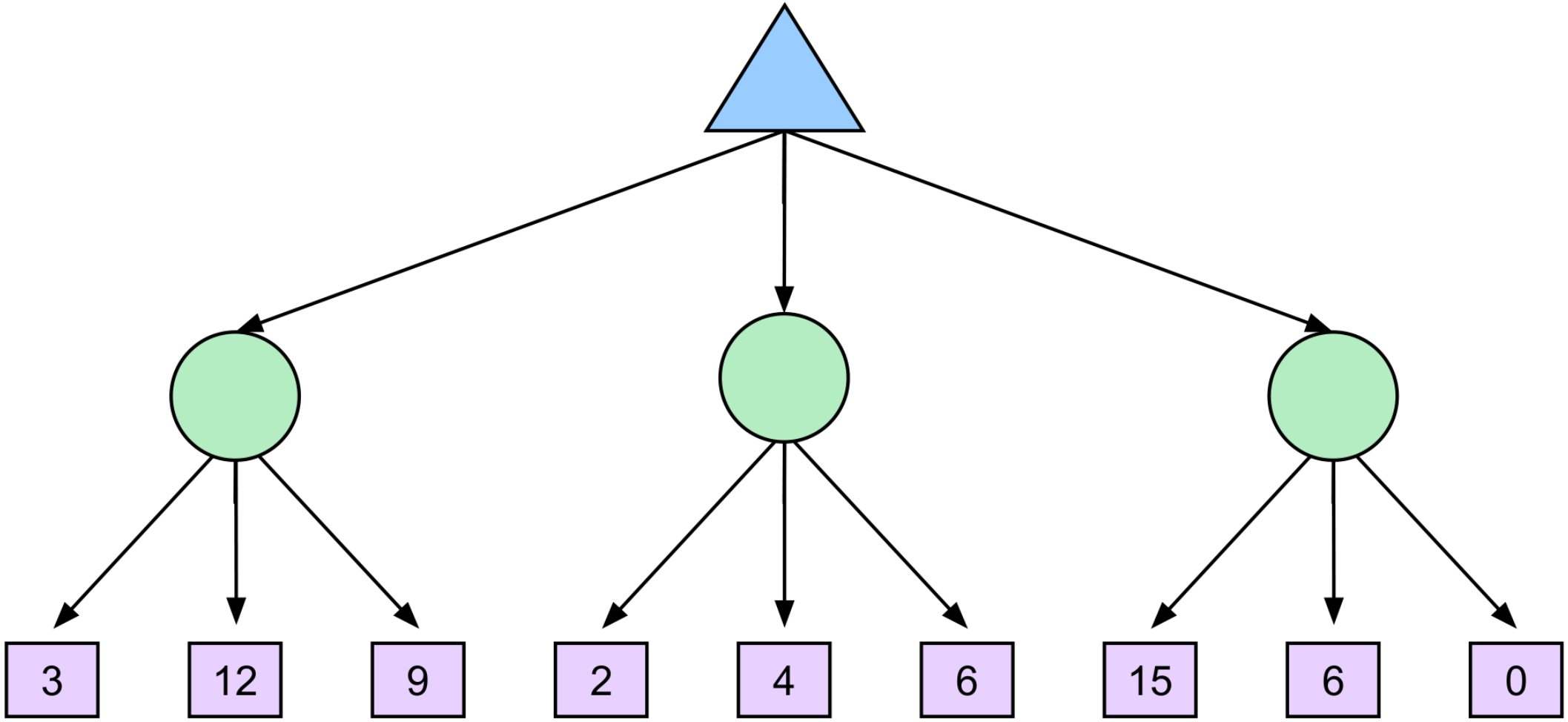
- Same dispatch as minimax; the **MIN case** is simply swapped for an **EXP case**.
- max-value starts at $-\infty$ and **rises**. exp-value starts at 0 and **accumulates**.
- That difference is why alpha-beta will not work here.

Expectimax: a Weighted Average

```
def exp-value(state):
    initialize v = 0
    for each successor of state:
        p = probability(successor)
        v += p * value(successor)
    return v
```



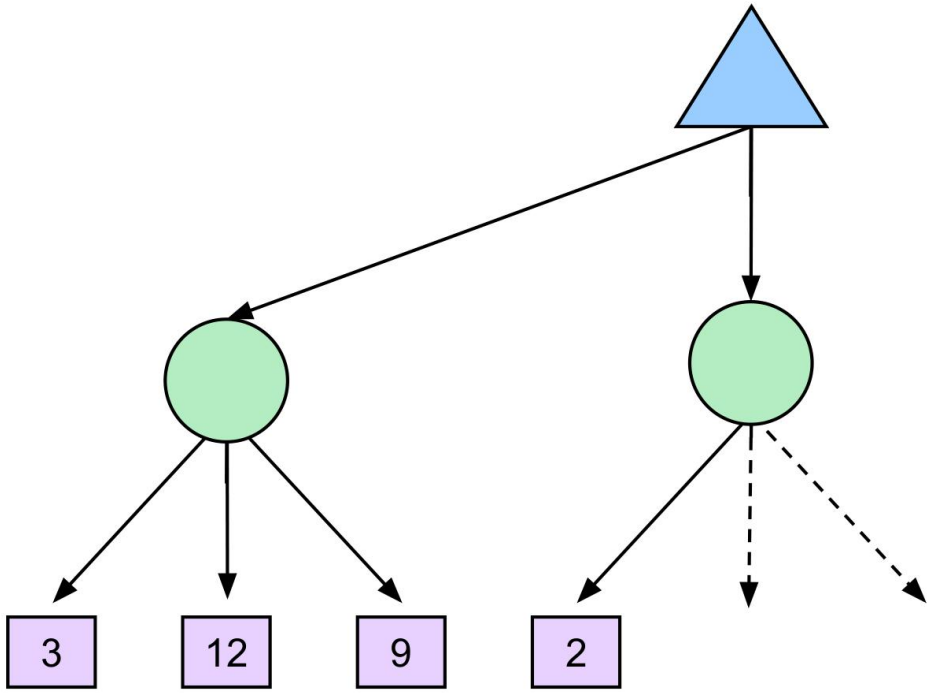
Expectimax Example

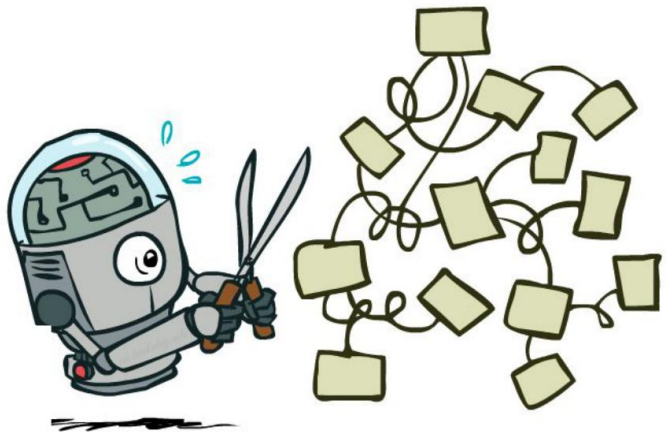


Expectimax Pruning?

- Can we prune as alpha-beta did?
- Alpha-beta relied on a MIN node **only ever falling**.
- A chance node has no such guarantee.

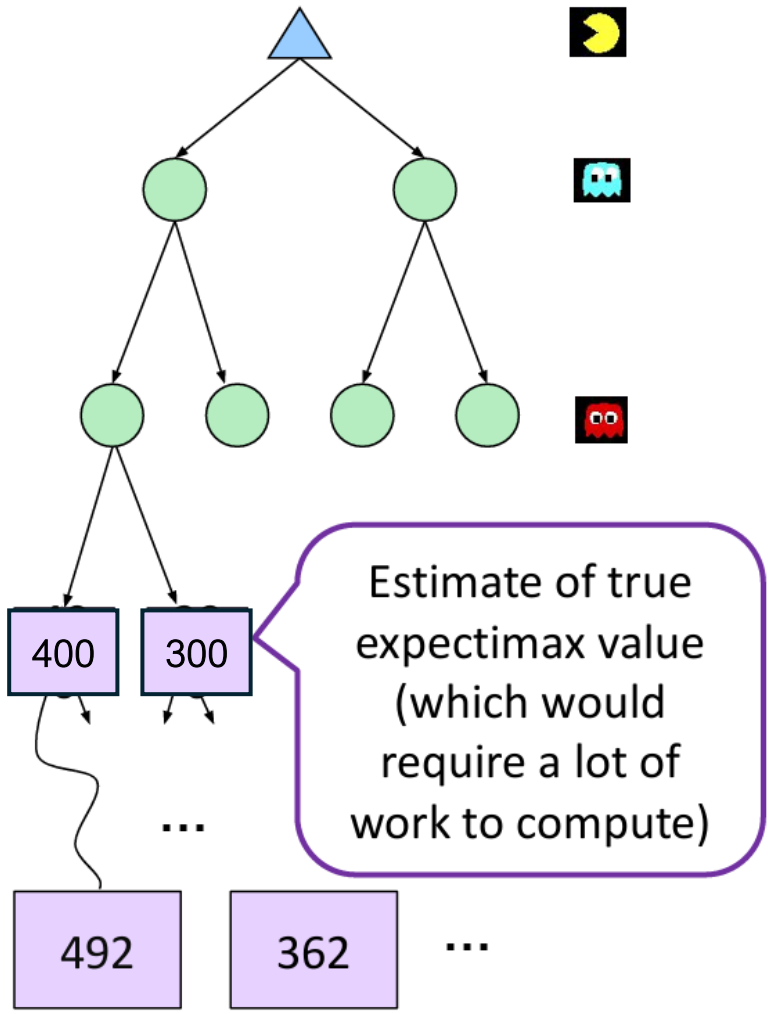
- Pruning come back only with **bounds on the magnitudes (all utility in 0 to 10)**, or **tiny probabilities** (0.999 you have already).





Depth-Limited Expectimax

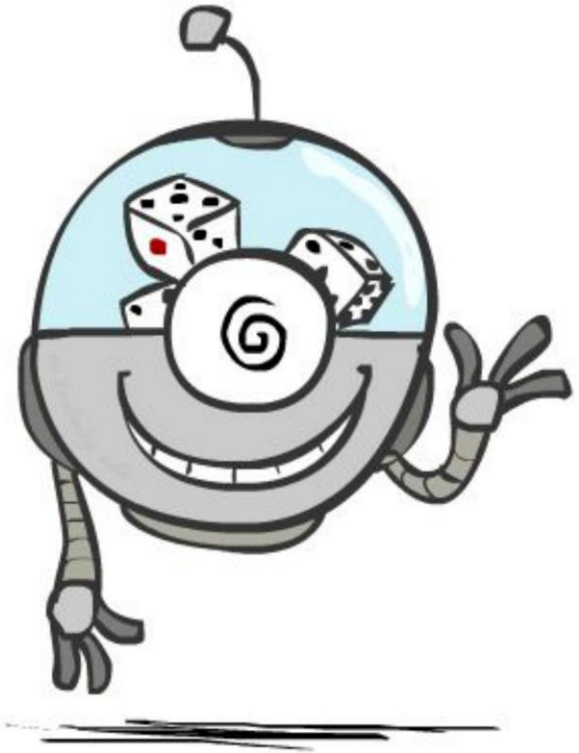
- Still too big, so we still stop early and still need an evaluation function
- It now estimates the expectimax value, not the minimax value
  - For Pac-Man: the average score from here on average at ghost nodes if they are random; minimize if they are adversarial
- You could learn it from played games, but ***you want the value under good play***.



2. Probabilities

The machinery the average needs, in one section

- Random variables and distributions
- **Expectation**: the weighted average, and why it is usually not one of the values
- Where the probabilities come from, and what they actually mean
- A worked modelling question of the kind that turns up in exams



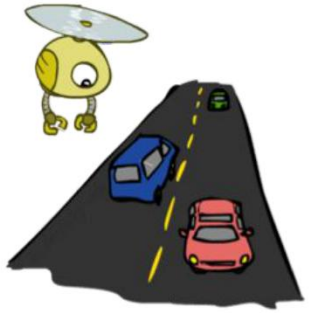
Reminder: Probabilities

- A ***random variable*** represents an event whose outcome is unknown.
- A ***probability distribution*** is an assignment of weights to outcomes.



0.25

- Example: Traffic on freeway
  - Random variable: T = whether there's traffic
  - Outcomes: T in {none, light, heavy}
  - Distribution: P(T=none) = 0.25, P(T=light) = 0.50, P(T=heavy) = 0.25



0.50

- Some laws of probability (more later):
  - Probabilities are always non-negative
  - Probabilities over all possible outcomes sum to one



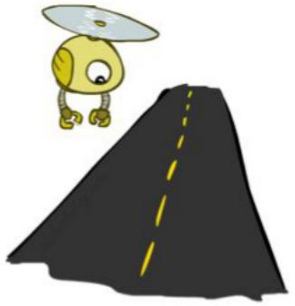
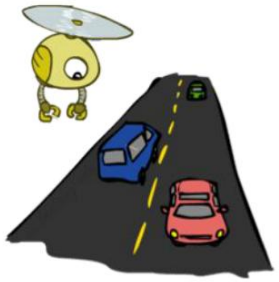
0.25

- As we get more evidence, probabilities may change:
  - P(T=heavy) = 0.25, P(T=heavy | Hour=8am) = 0.60
  - We'll talk about methods for reasoning and updating probabilities later

Reminder: Expectations

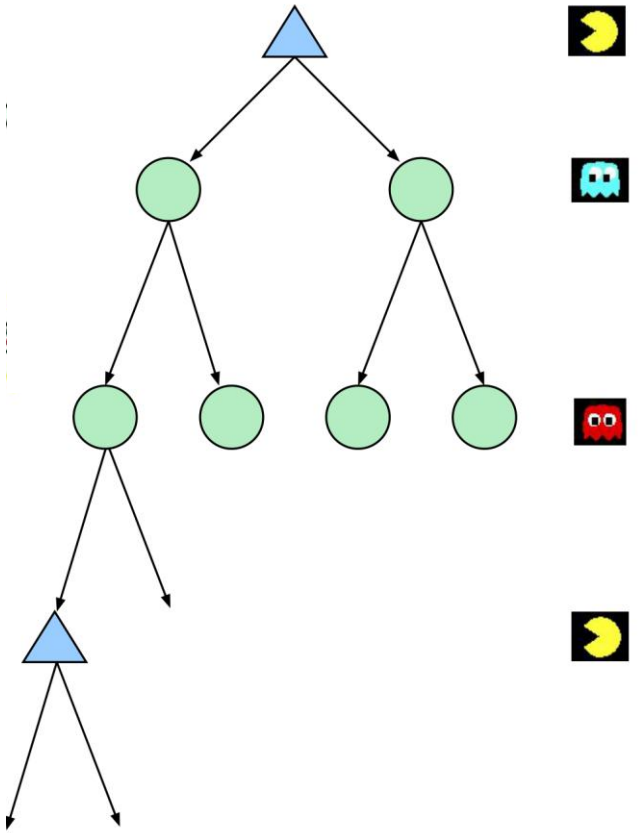
- The expected value is the average, weighted by the distribution
- How long to get to the airport?
  - (0.25)(20) + (0.50)(30) + (0.25)(60) = 35 min
- You will never experience a 35 minute trip. It is not one of the three
- It is the number to plan with, and what a chance node computes

Time:	20 min	+	30 min	+	60 min		
	x		x		x		
Probability:	0.25		0.50		0.25		35 min


What Probabilities to Use?

- In expectimax search, we have ***a probabilistic model*** of how the opponent (or environment) will behave in any state.
  - Model could be a simple uniform distribution: rolling a die
  - Model could be sophisticated and require a great deal of computation
  - We have a chance node for any outcome out of our control: opponent or environment
  - The model might say that adversarial actions are likely!

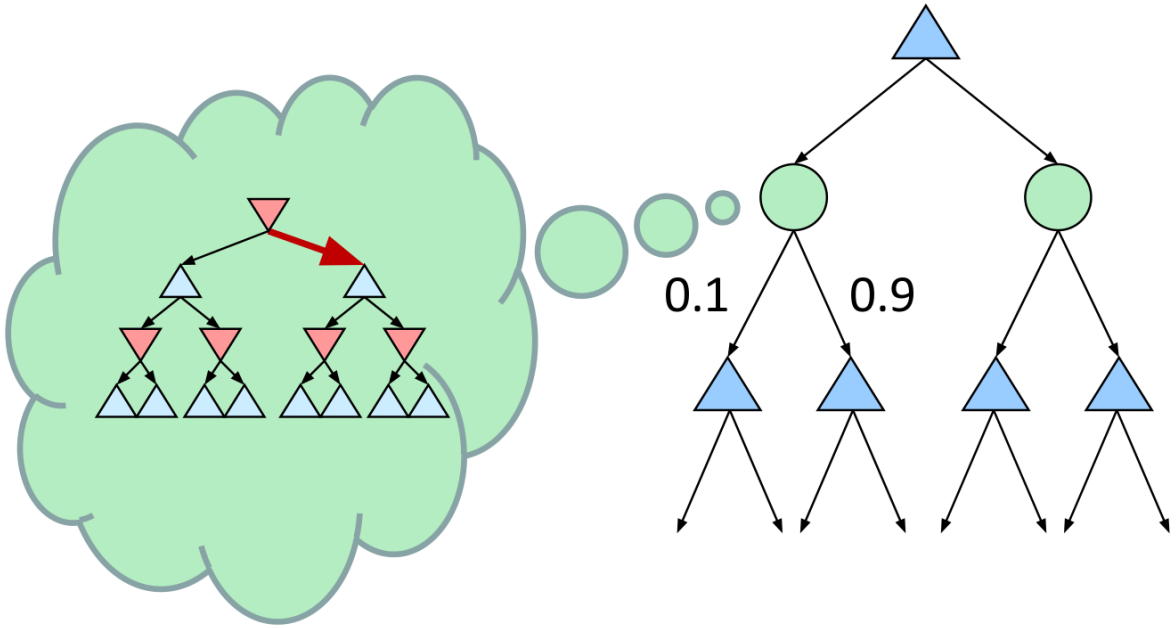


- For now, the probabilities arrive attached magically. Where they come from is later.

A probabilistic belief about another agent does not mean that the agent is flipping coins.

Quiz: Informed Probabilities

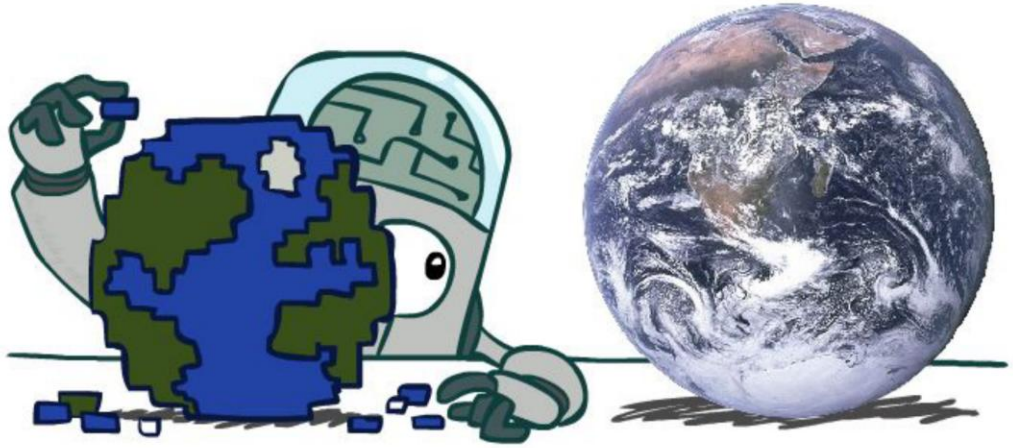
- Let's say you know that your opponent is actually running a depth 2 minimax, using the result 80% of the time, and moving randomly otherwise
- Question: What tree search should you use?
- Answer: **Expectimax**.
  - Minimax is the special case where the opponent is optimal with probability 1
  - So, expectimax is the generalization, not the alternative.
- The cost: each chance node needs you to simulate your opponent
  - Worse if they are simulating you at the same time



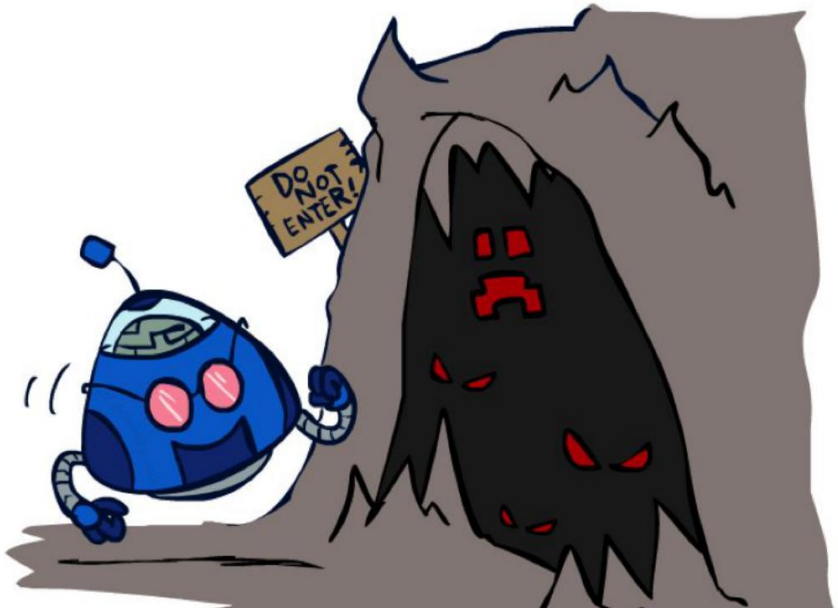
3. Modeling Assumptions

Getting the algorithm right is not the same as getting the world right

- **Dangerous optimism** and **dangerous pessimism**, side by side
- The 2 x 2 experiment: two Pac-Man agents against two kinds of ghost
- Why the two ways of being wrong **do not cost the same**



The Dangers of Optimism and Pessimism



Dangerous Optimism

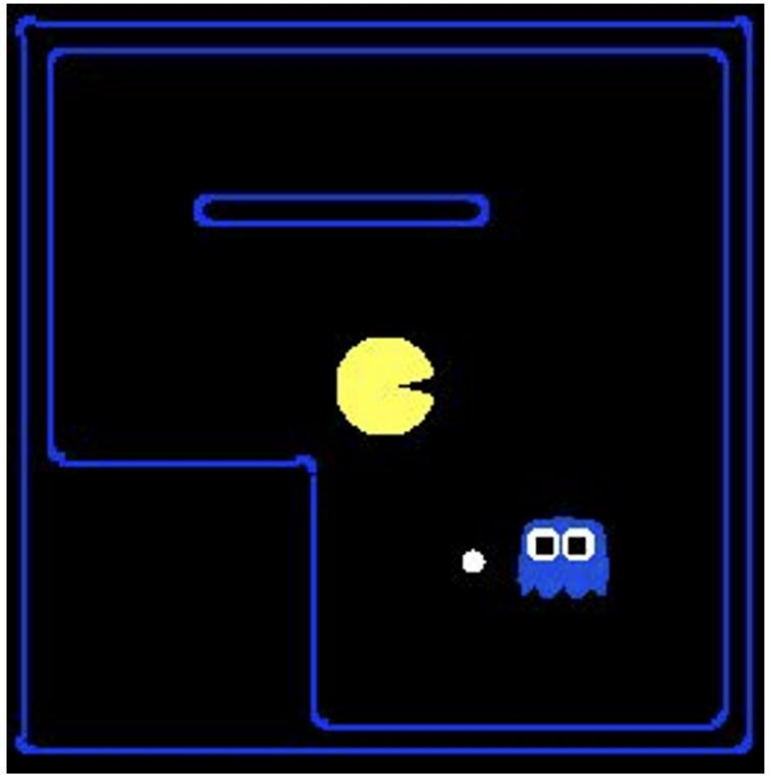
- Assuming chance when the world is **adversarial**
- You treat the bad outcomes as unlucky, so you walk into them.
- Worse: an adversary will **arrange** exactly the situations you dismissed as unlikely



Dangerous Pessimism

- Assuming the worst case when it is **not likely**
- You run **away from everything** and give up value for safety.
- The robot that is terrified of a rabbit

Assumptions vs. Reality



	Adversarial Ghost	Random Ghost
Minimax Pacman	Won 5/5 Avg. score **483**	Won 5/5 Avg. score **493**
Expectimax Pacman	Won 1/5 Avg. score **-303**	Won 5/5 Avg. score **503**

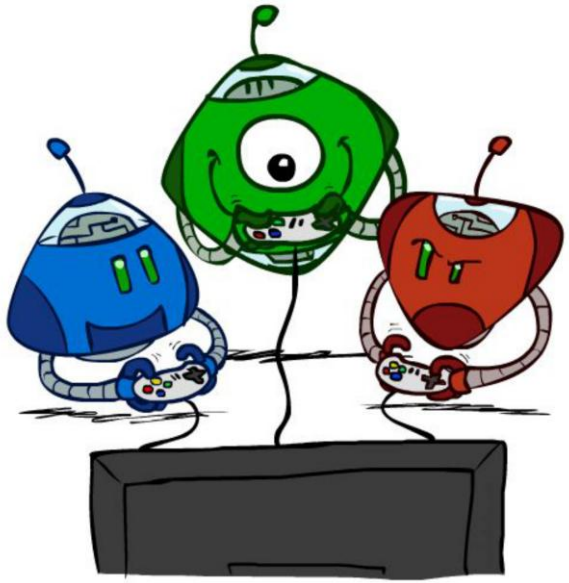
Results from playing 5 games

Pacman used depth 4 search with an eval function that avoids trouble.
Ghost used depth 2 search with an eval function that seeks Pacman.

4. Other Game Types

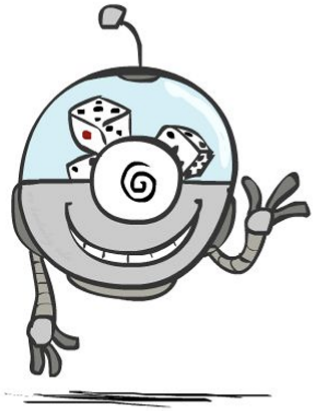
Lifting the assumptions, one at a time, and finding that everything still works

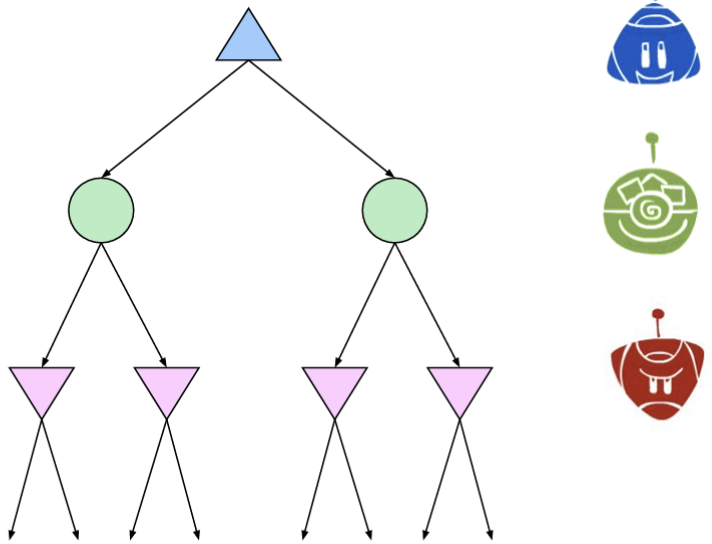
- **Mixed layers**: max, min and chance in the same tree
- **Monopoly**, and why a wide chance layer changes the arithmetic
- **More than two players**, and games that are not zero-sum
- Utility **tuples**, and where cooperation comes from



Mixed Layer Types

- There is no rule you putting max, min and chance layers in one tree
- The name is ***expectiminimax***,
- The environment is an extra player that is not adversarial
- Then each node just does what its own layer calls for





Example: Monopoly

- Expectiminimax
  - Environment is an extra “random agent” player: the dice roll that comes after each min/max agent’s move
  - MAX decides (buy / build / pass) → dice decide where the opponent lands → MIN decides → dice again …
  - Each node computes the appropriate combination of its children



Multi-Agent Utilities

- What if the game is ***not zero-sum***, or has ***more than two players***?
- Terminals carry a utility ***tuple***, one entry per player. Node values too
- Each player maximizes its own component
- There is **no minimizer** anywhere in this tree. Everyone maximizes
- Zero-sum was the special case where one number was enough
- **Cooperation** and **competition** appear by themselves dynamically

